

2026 年高教社杯全国大学生数学建模竞赛

A 题 药材的烘干问题

摘 要

本文针对中药材热风干燥的四个问题，建立了圆柱轴对称热质耦合传递模型，并给出离散通量严格闭合的数值求解方法。

针对问题 1，采用附录 2 的常物性参数与经验扩散系数 $D = 7 \times 10^{-9} e^{-0.89/C}$ ，建立圆柱径向一维热传导方程与水分扩散方程，表面取第三类（Robin）边界条件，烘房温度与水分浓度由附件 1 时变数据线性插值给出。用守恒型有限体积离散与 Crank–Nicolson 格式求解，得 1800 s 时表面温度由初值 28 °C 升至 36.79 °C，表面水分浓度由 2.55 kg/kg 降至 1.51 kg/kg，水分变化集中于近表面约 5 mm 的边界层内，中心含水率几乎不变，符合预热平衡阶段的物理特征。

针对问题 2，采用附录 3 的经验公式，其中密度、比热容、导热系数随水分浓度 C 变化，扩散系数 $D(C, T) = 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T}$ 随 C 与绝对温度 T 变化，建立热质双向弱耦合模型：温度场通过 Arrhenius 因子影响水分扩散，水分场通过物性参数影响温度场；求解 0–3 h 的温度与水分场（表 3、表 4）。**针对问题 3**，沿用该模型，烘房条件取附件 1 插值（ $t \leq 14400$ s）与恒温段参数（50.165 °C, 0.04986 kg/kg）两段拼接，按判据各处水分浓度 ≤ 0.15 kg/kg 求得烘干时长 $t^* = 205701$ s，即 **57.14 h**（约 2.38 天），处于题述烘干过程一般持续 2–3 天范围内。

针对问题 4，考虑药材尺寸收缩：采用附录 4 的经验公式，并以附件 2 的半径数据经 Pchip 保形插值构造动边界 $R(t)$ ；引入随体坐标 $\zeta = r/R(t)$ ，将热传导与水分扩散方程变换为无对流项的 Lagrange 形式，在同一守恒离散框架下求解。求得含收缩时的烘干时长 $t^* = 182897.5$ s，即 **50.80 h**（约 2.12 天）。

数值方法与检验：空间采用离散通量严格闭合的有限体积离散（表面节点位于真实表面，Robin 通量与内部界面通量逐时间层对消），时间采用 Crank–Nicolson 格式，非线性与双向耦合用交错 Picard 迭代处理。模型通过四重检验：与 Bessel 级数解析解对照，温度场四位小数完全一致；空间步长减半收敛比达理论值 4（问题 1、2 实测比值 4.0），验证二阶精度；离散质量守恒闭合差在 10^{-7} 以内（问题 1 达机器精度）；长时程渐近精确收敛到烘房参数。此外对水分汽化潜热作数量级分析：表面蒸发吸热与对流显热之比约 3–5，其不确定性以时长敏感性区间（约 +40% ~ 50%，见 7.6 节）计入结果解读。

关键词：热风干燥；热质耦合传递；圆柱轴对称扩散；Crank–Nicolson 格式；移动边界；有效水分扩散系数

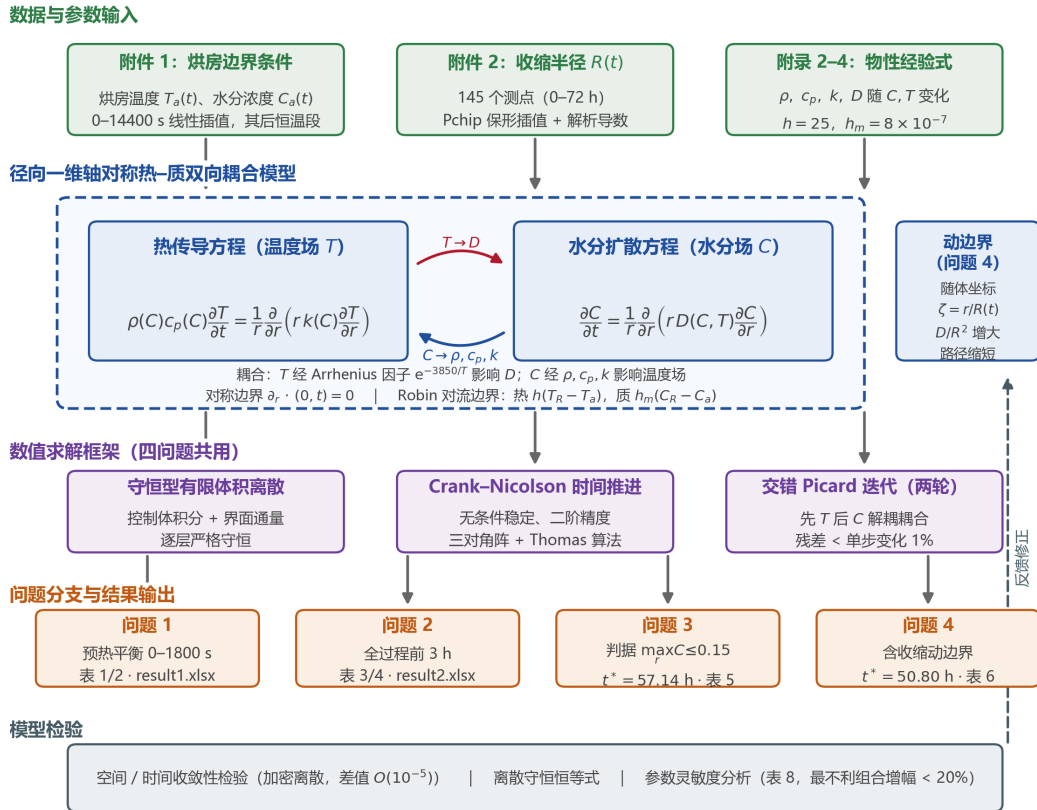


图 1: 架构图

一、问题重述

中药材热风干燥通常包含预热平衡与恒温干燥两个阶段，通过调控烘房温湿环境完成干燥。某中药材形状近似为圆柱形，长 25 cm，半径 2 cm，初始温度 28 °C，初始水分浓度（干基含水率）2.55 kg/kg。需建立数学模型研究如下四个问题：

问题 1：附件 1 给出烘干初期烘房温度与水分浓度的时变数据，附录 2 给出相关参数。建立预热平衡阶段药材温度与水分浓度变化规律的数学模型，按表 1、表 2 格式给出 100–1800 s 内 7 个时刻、距中心 0–2 cm 处的结果，并给出 1800 s 内每隔 1 s、距离每隔 0.1 cm 的完整结果。

问题 2：整个烘干过程包含参数不同的预热平衡与恒温干燥两阶段，经验公式统一采用附录 3。建立整个烘干过程的数学模型，按表 3、表 4 格式给出前 3 h 内每隔 0.5 h、距中心 0–2 cm 处的结果，并给出每隔 1 s、距离每隔 0.1 cm 的完整结果。

问题 3：按烘干要求，药材各处水分浓度应低于 0.15 kg/kg。确定烘干所需时间，按表 5 格式给出每隔 6 h、距中心每隔 0.5 cm 处的水分浓度，并给出每隔 60 s、距离每隔 0.1 cm 的完整结果。

问题 4：实际烘干中药材因水分流失发生尺寸变化。根据附件 2 的半径数据与附录

4 的经验公式，确定烘干时长，按表 6 格式给出每隔 6 h、距中心每隔 0.5 cm 处及药材表面处的水分浓度，并给出每隔 60 s、距离每隔 0.1 cm 的完整结果。

所有结果保留四位小数。

二、问题分析

2.1 几何与传递机理分析

药材为长 25 cm、半径 2 cm 的细长圆柱，长径比达 6.25。轴向与径向的扩散特征时间之比约为 $(L/(2R))^2 \approx 39$ （轴向特征尺度取半长 $L/2$ ，即到最近端面的距离），故端面效应与轴向不均匀性可以忽略，药材内部的温度与水分梯度主要由径向热风对流引起。将药材视为无限长圆柱，取径向一维轴对称模型即可（文献中圆柱形食品与药材的干燥均广泛采用径向一维模型 [6, 7]）。周向按轴对称处理，初始条件均匀，故解不依赖周向角。

内部传递机理：热量按傅里叶定律自表面向内传导 [3, 12]；水分在浓度梯度驱动下按菲克定律向表面扩散，在表面以对流方式传递到烘房空气。表面边界均为第三类（Robin）条件：热侧由对流换热系数 h 描述，质侧由对流传质系数 h_m 描述。

2.2 热质耦合的机制分析

本题的耦合是**双向弱耦合**：温度场影响水分场——扩散系数经验式 $D(C, T)$ 含 Arrhenius 因子 $e^{-3850/T}$ ，温度升高显著加速水分扩散；水分场影响温度场——附录 3、4 中密度、比热容、导热系数均随水分浓度 C 变化，水分流失改变热容与热阻。两个场通过物性参数相互影响，而边界条件本身不直接交叉（无 Soret/Dufour 交叉效应）。数值上采用交错 Picard 迭代：同一时间步内先由当前水分场求温度场，再用新温度场求水分场，交错循环直至满足 5.5 节的收敛判据（各算例平均 2.2–3.4 轮，最多 4 轮）。

关于水分汽化的局部吸热效应：严格意义下蒸发吸热会降低表面温度，但本题题给的经验式（ $D(C, T)$ 、 h_m 等）是由干燥实验数据拟合得到的**有效参数**，其数值可能已隐含吸收汽化与吸附等微观机制的净效应——现有信息不足以唯一判定其吸收程度，但若在题给经验式之上再显式加入潜热汇项，将面临与经验参数重复计账的风险，且需引入题给之外的饱和蒸气压等参数。这一判断的数量级依据在 7.6 节给出专门分析（时长敏感性区间约 +40% ~ 50%），并在模型假设 H5 中明确声明。

2.3 两阶段烘房条件的处理

附件 1 给出烘干初期（0–14400 s）烘房温度与水分浓度的时变序列（共 241 个数据点）。整个烘干过程分为预热平衡与恒温干燥两阶段：预热平衡段烘房条件按附件 1

线性插值；其后进入恒温段，烘房参数取附件 1 末值并保持恒定，即 $T_a = 50.165\text{ }^\circ\text{C}$ ， $C_a = 0.04986\text{ kg/kg}$ 。问题 1 的时间范围（0–1800 s）与问题 2 的输出范围（前 3 h）均落在附件 1 数据段内，直接插值即可；问题 3、4 的求解需贯穿全程，采用上述两段拼接的边界条件。

2.4 四个问题的建模差异

四个问题在物性经验式、烘房条件、几何处理与输出要求上的差异是本题的关键，逐一对照如下：

条件	问题 1	问题 2	问题 3	问题 4
物性经验式	附录 2	附录 3	附录 3	附录 4
D 是否随 T 变	否（仅随 C ）	是	是	是
ρ, c_p, k 是否随 C 变	否（常数）	是	是	是
烘房条件	附件 1	附件 1	附件 1+ 恒温段	附件 1+ 恒温段
几何收缩（附件 2）	否	否	否	是（ $R(t)$ ）
求解时长	1800 s	前 3 h	至 $C \leq 0.15$	至 $C \leq 0.15$
输出	表 1/2	表 3/4	表 5、 t^*	表 6、 t^*

三、模型假设

- H1. 连续介质近似。药材内部的多孔骨架、细胞间隙与水分经体积平均视为连续介质，温度与水分浓度在宏观尺度上连续分布，可用傅里叶定律与菲克定律描述。这一做法是多孔介质干燥理论的标准框架 [4, 5]。
- H2. 径向一维轴对称。长径比 6.25，忽略轴向传递与端面效应，忽略周向不均匀性，温度与水分浓度仅为 r （或 ζ ）与 t 的函数。
- H3. 初值均匀。 $T(r, 0) = 28\text{ }^\circ\text{C}$ ， $C(r, 0) = 2.55\text{ kg/kg}$ 。
- H4. 对流系数恒定。 $h = 25\text{ W}/(\text{m}^2\text{K})$ 、 $h_m = 8 \times 10^{-7}\text{ m/s}$ 在全程视为常数。文献研究表明，常规对流干燥中对流系数在恒速段与降速段内近似为常数，仅在段间过渡处显著变化 [5, 10]，故该简化合理。
- H5. 经验式的有效参数观点（含汽化潜热）。题给的 ρ, c_p, k, D, h_m 是由实验标定的有效参数： $D(C, T)$ 的 Arrhenius 指数对应的活化能 $E_a = 32.0\text{ kJ/mol}$ 高于纯液态水自扩散的活化能（约 18–20 kJ/mol[17]），其差值对应解吸与汽化的综合能垒，说明潜热效应可能已被有效参数部分吸收，但现有信息不足以唯一判定其吸收程度（数量级分析见 7.6 节）。因此主模型采用显热传导与有效水分扩散系数，不显式引入蒸

发潜热汇项：若显式叠加，将面临与经验参数重复计账的风险，且需引入题给之外的饱和蒸气压等数据。

- H6. **问题 1–3 忽略收缩。**题面仅要求问题 4 依据附件 2 考虑尺寸变化，且附录 2、3 的物性参数不含收缩机制，故问题 1–3 取固定半径 $R = 2 \text{ cm}$ 。附件 2 数据显示前 30 min 半径收缩约 6.4%，全程收缩约 40.1%，该简化对问题 1–3 的影响在问题 4 中通过含收缩模型评估（见 6.4 节对比）。
- H7. **烘房环境均匀且按题给数据变化。**预热平衡段按附件 1 线性插值；恒温干燥段取附件 1 末值 ($50.165 \text{ }^{\circ}\text{C}$, 0.04986 kg/kg) 恒定——14400 s 后固定为附件 1 末值是对题给信息的重要外延（附件 1 仅覆盖 0–14400 s），其对结论的影响由 7.7 节的参数敏感性分析评估。
- H8. **收缩各向同性。**问题 4 中半径按附件 2 收缩，且径向收缩均匀（水分浓度与温度仅沿径向变化，与题给一维数据自洽）；干基含水率定义于固相骨架，随体坐标下无对流项（见 5.4 节）。

四、符号说明

符号	含义	符号	含义
r	径向坐标 (m)	t	时间 (s)
R	药材半径 (m, 问题 4 中为 $R(t)$)	ζ	随体坐标 $r/R(t)$
T	药材温度 ($^{\circ}\text{C}$)	T_a	烘房温度 ($^{\circ}\text{C}$)
C	水分浓度/干基含水率 (kg/kg)	C_a	烘房水分浓度 (kg/kg)
ρ	密度 (kg/m^3)	c_p	比热容 ($\text{J}/(\text{kg}\cdot\text{K})$)
k	导热系数 ($\text{W}/(\text{m}\cdot\text{K})$)	D	水分扩散系数 (m^2/s)
h	对流换热系数 ($\text{W}/(\text{m}^2\cdot\text{K})$)	h_m	对流传质系数 (m/s)
T_0, C_0	初始温度/水分浓度	t^*	烘干时长 (h)
$\Delta r, \Delta t$	空间/时间步长	N	径向节点数
Bi	毕渥数 hR/k	E_a	表观活化能 (kJ/mol)
ΔH_v	水的汽化潜热 (J/kg)	RH	空气相对湿度

五、模型的建立与求解

5.1 问题 1：预热平衡阶段的模型

预热平衡阶段药材尚处低温（初温 $28 \text{ }^{\circ}\text{C}$ ），物性可按附录 2 取常数： $\rho = 820 \text{ kg/m}^3$, $c_p = 2600 \text{ J}/(\text{kg}\cdot\text{K})$, $k = 0.36 \text{ W}/(\text{m}\cdot\text{K})$, $h = 25 \text{ W}/(\text{m}^2\cdot\text{K})$, $h_m = 8 \times 10^{-7} \text{ m/s}$ ；水分

扩散系数采用经验式

$$D(C) = 7 \times 10^{-9} e^{-0.89/C} \quad (\text{m}^2/\text{s}). \quad (1)$$

径向一维轴对称模型的控制方程、定解条件为

$$\rho c_p \frac{\partial T}{\partial t} = \frac{k}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right), \quad 0 < r < R, \quad t > 0, \quad (2)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C) \frac{\partial C}{\partial r} \right), \quad 0 < r < R, \quad t > 0, \quad (3)$$

$$\frac{\partial T}{\partial r}(0, t) = 0, \quad -k \frac{\partial T}{\partial r}(R, t) = h(T(R, t) - T_a(t)), \quad (4)$$

$$\frac{\partial C}{\partial r}(0, t) = 0, \quad -D \frac{\partial C}{\partial r}(R, t) = h_m(C(R, t) - C_a(t)), \quad (5)$$

$$T(r, 0) = T_0 = 28 \text{ }^\circ\text{C}, \quad C(r, 0) = C_0 = 2.55 \text{ kg/kg}. \quad (6)$$

其中 $T_a(t)$ 、 $C_a(t)$ 由附件 1 线性插值；附件 1 的 C_a 单位为 kg/kg、与干基含水率 C 同纲，本文将其解释为烘房空气状态对应的药材等效平衡水分浓度，故 Robin 边界驱动力 $C_R - C_a$ 直接成立（按空气含湿比解释的情景分析见 7.6 节）。式 (2)、(3) 在 $r = 0$ 处按对称性与洛必达法则化为 $\frac{\partial T}{\partial t} = 2\alpha \frac{\partial^2 T}{\partial r^2}$ 及 $\frac{\partial C}{\partial t} = 2 \frac{\partial}{\partial r} \left(D(C) \frac{\partial C}{\partial r} \right)$ ，其中 $\alpha = k/(\rho c_p)$ 为热扩散率。

5.2 问题 2：0–3 h 温度与水分场模型

物性按题设统一采用附录 3 的经验式：

$$\rho(C) = 650 + 128C, \quad c_p(C) = 1450 + 2736 \frac{C}{C+1}, \quad k(C) = 0.21 + 0.38 \frac{C}{C+1}, \quad (7)$$

$$D(C, T) = 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T} \quad (\text{m}^2/\text{s}), \quad (8)$$

其中 T 为绝对温度 (K)。此时热传导方程中的 ρc_p 与 k 均随 C 变化，且 D 同时依赖 C 与 T ，两场构成双向弱耦合（边界条件不直接交叉，且忽略水分输运携带的显焓等交叉项，见 2.2 节）：

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial T}{\partial r} \right), \quad (9)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right), \quad (10)$$

边界条件与式 (4)、(5) 形式相同，其中质侧条件的 D 取表面值 $D(C_R, T_R)$ 。题面未另行给出对流系数， h 、 h_m 沿用附录 2 取值（ $h = 25 \text{ W}/(\text{m}^2\text{K})$ ， $h_m = 8 \times 10^{-7} \text{ m/s}$ ），与假设 H4 一致。初值同式 (6)。

烘房条件 $T_a(t)$ 、 $C_a(t)$ 按 2.3 节由附件 1 线性插值给出，两场耦合机制见 2.2 节，求解结果按表 3、表 4 输出。

5.3 问题 3：全程烘干模型与时长判据

问题 3 沿用式 (9)–(10) 的同一方程组与初边值（物性及 h 、 h_m 同 5.2 节），但需求解整个烘干过程，时间范围远超附件 1 的 14400 s，故烘房条件按两阶段拼接：

$$T_a(t) = \begin{cases} \text{附件 1 线性插值,} & t \leq 14400 \text{ s,} \\ 50.165 \text{ }^\circ\text{C,} & t > 14400 \text{ s,} \end{cases} \quad C_a(t) = \begin{cases} \text{附件 1 线性插值,} & t \leq 14400 \text{ s,} \\ 0.04986 \text{ kg/kg,} & t > 14400 \text{ s.} \end{cases} \quad (11)$$

即前 14400 s 按附件 1 线性插值，其后进入恒温段，烘房参数取附件 1 末值并保持恒定。按题面判据求解：当各处水分浓度 $\leq 0.15 \text{ kg/kg}$ 即 $\max_r C(r, t) \leq 0.15 \text{ kg/kg}$ 首次成立时停止，对应时刻即烘干时长 t^* ，全程水分场按表 5 输出。

5.4 问题 4：含收缩的模型

(1) 动边界与随体坐标。附件 2 给出每隔 1800 s 的药材半径数据（共 145 点，覆盖 0–72 h，自 67 h 起恒定于 1.198 cm）。为得到光滑的 $R(t)$ ，采用保形分段三次 Hermite (Pchip) 插值（保单调、无过冲），并取 dR/dt 为其解析导数。收缩使外边界随物料移动，故将径向坐标换为随体 (Lagrange) 坐标

$$\zeta = \frac{r}{R(t)} \in [0, 1], \quad (12)$$

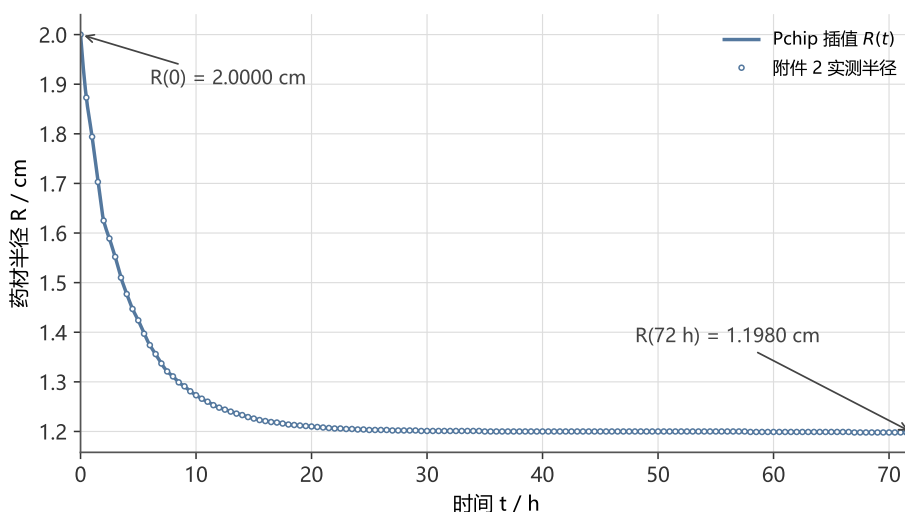


图 7 问题四药材半径随时间的收缩曲线

说明：附件 2 的半径数据及 Pchip 插值结果，对应式 (12) 中的动边界 $R(t)$ 。

各向同性且径向均匀的收缩（假设 H8）意味着任一物质点按统一比例 $R(t)/R(0)$ 收缩，即物质点位置 $r_p(t) = \zeta_p R(t)$ 、径向速度 $v_r = \zeta_p R'(t) = (R'/R) r_p$ ，故 ζ 恒标记同一干物质质点。下面从守恒律推导随体坐标下的方程：干基含水率 C 定义于固相骨架，对随

骨架运动的物质元，干固体质量守恒要求 $dM_d = \rho_d dV$ 恒定 ($\rho_d = \rho/(1+C)$ 为干物质密度)，水分守恒给出 $dM_d DC/Dt = -\nabla \cdot q_d dV$ ，其中 $q_d = -\rho_d D \nabla C$ 为相对骨架的 Fick 通量 [4]， D/Dt 为随体导数。联立两式并利用干固体守恒恒等式 $\partial \rho_d / \partial t + \nabla \cdot (\rho_d v) = 0$ ，对流项 $v \cdot \nabla C$ 与 ρ_d 的变化率项严格相消，得 $\rho_d DC/Dt = \nabla \cdot (\rho_d D \nabla C)$ ——这正是随体坐标无对流项的由来。在题给有效参数观点下（假设 H5）， ρ_d 随 C 的变化已隐含于 $D(C, T)$ 的经验标定中，取 ρ_d 均匀近似即得式 (13) 的水分方程；热量方程经同样的随体变换并忽略水分携焓交叉项（见 2.2 节）化为

$$\left. \frac{\partial C}{\partial t} \right|_{\zeta} = \frac{1}{R^2(t) \zeta} \frac{\partial}{\partial \zeta} \left(\zeta D(C, T) \frac{\partial C}{\partial \zeta} \right), \quad (13)$$

$$\rho(C) c_p(C) \left. \frac{\partial T}{\partial t} \right|_{\zeta} = \frac{1}{R^2(t) \zeta} \frac{\partial}{\partial \zeta} \left(\zeta k(C) \frac{\partial T}{\partial \zeta} \right), \quad (14)$$

物性由附录 4 给出： $\rho = 760 + 90C$ ， $c_p = 1850 + 2150 C/(C+1)$ ， $k = 0.12 + 0.20 C/(C+1)$ ，

$$D(C, T) = 4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T} \quad (\text{m}^2/\text{s}). \quad (15)$$

边界条件 ($\zeta = 1$ 为真实表面)：

$$-\frac{k}{R} \frac{\partial T}{\partial \zeta}(1, t) = h(T(1, t) - T_a(t)), \quad -\frac{D}{R} \frac{\partial C}{\partial \zeta}(1, t) = h_m(C(1, t) - C_a(t)), \quad (16)$$

对称条件 $\frac{\partial T}{\partial \zeta}(0, t) = \frac{\partial C}{\partial \zeta}(0, t) = 0$ 。表 6 中到药材中心的距离列在 $d > R(t)$ 时该位置已在药材之外，表中以“—”表示（相应单元格按模板留空）；药材表面列即 $\zeta = 1$ 处的值。固定距离列在 ζ 坐标的离散节点上经 Pchip 插值取值（插值阶的选择依据见 7.2 节）。

(2) 收缩对烘干的影响机制。由式 (13) 可见收缩的加速作用是同一几何效应的两种等价表述： $R(t)$ 减小使算子系数 D/R^2 增大（全程 R 由 2 cm 减至 1.198 cm， R^{-2} 增大 2.8 倍），其物理含义即浓度场随骨架向内收缩、等效扩散路径缩短，两者并非相互独立的机制。附录 4 的 D 前因子与指数均小于附录 3，收缩效应与物性差异的综合影响在 6.4 节对比。

5.5 数值方法

四个问题共用一套守恒型离散框架，下面以问题 1 的径向空间离散（问题 4 的 ζ 坐标离散同理）说明。

(1) 空间离散节点与控制体。取节点 $r_i = i\Delta r$ ($i = 0, 1, \dots, N-1$)，则表面节点恰位于真实表面： $r_{N-1} = R$ 。每个节点 i 对应控制体体积 $v_i = \int_{CV_i} r dr$ ：内部节点取 $[r_{i-1/2}, r_{i+1/2}]$ ，圆心节点取 $[0, r_{1/2}]$ ，表面节点取 $[r_{N-3/2}, R]$ 。方程 (2)–(3) 在控制体上积分（对 $r dr$ ），通量在界面 $i \pm 1/2$ 用算术平均的扩散系数（如 $D_{i-1/2} = \frac{1}{2}(D_{i-1} + D_i)$ ）离散，得到

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{1}{2}(Lu^{n+1} + Lu^n)_i + \frac{1}{2}(s^{n+1} + s^n)_i, \quad (17)$$

其中 u 代表 T 或 C , $(Lu)_i$ 为界面通量差

$$(Lu)_i = \left[r_{i+1/2} K_{i+1/2} \frac{u_{i+1} - u_i}{\Delta r} - r_{i-1/2} K_{i-1/2} \frac{u_i - u_{i-1}}{\Delta r} \right] / v_i, \quad (18)$$

圆心行按对称性与洛必达法则取 $4K_{1/2}(u_1 - u_0)/\Delta r^2$ ($v_0 = \Delta r^2/8$)。表面控制体的体积为 $v_{N-1} = ((N-1)^2 - (N - \frac{3}{2})^2)\Delta r^2/2$, 其外界面即真实表面, 将 Robin 条件 (4)、(5) 直接作为外界面通量作用于表面节点:

$$(Lu)_{N-1} = \left[r_{N-3/2} K_{N-3/2} \frac{u_{N-2} - u_{N-1}}{\Delta r} - R\beta K_s (u_{N-1} - u_a) \right] / v_{N-1}, \quad (19)$$

其中热方程 $\beta = h/k_s$, 质方程 $\beta = h_m/D_s$ 。式 (19) 中表面节点值即真实表面值, 通量取 $Rh(T_R - T_a)$ 与 $Rh_m(C_R - C_a)$, 与内部界面通量逐时间层严格对消, 保证离散守恒; 各离散项截断误差的泰勒展开分析见本节 (5)。

(2) 时间推进与线性代数。 式 (17) 为 Crank–Nicolson 格式 [1] (无条件稳定、时间二阶), 每步化为三对角方程组, 用追赶法 (Thomas 算法) 求解: 对三对角方程组 $a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$, 前代 $c'_0 = c_0/b_0$, $d'_0 = d_0/b_0$, 随后 $m = b_i - a_i c'_{i-1}$, $c'_i = c_i/m$, $d'_i = (d_i - a_i d'_{i-1})/m$; 回代 $x_{N-1} = d'_{N-1}$, $x_i = d'_i - c'_i x_{i+1}$, 复杂度 $O(N)$ 。空间通量项 L 由式 (18)、(19) 给出, 时间层上源项 (边界贡献) 取 n 、 $n+1$ 两层的算术平均。

(3) 非线性与耦合的处理。 附录 3、4 的物性随 C, T 变化, 且两场双向耦合, 采用交错 Picard 迭代: 时间步 $[t_n, t_{n+1}]$ 内,

- (i) 用当前水分场 C^k 计算 $k(C)$ 、 ρc_p , 求解温度场得 T^{k+1} ;
- (ii) 用 C^k 与温度 T^{k+1} 计算 $D(C, T)$, 求解水分场得 C^{k+1} ;
- (iii) 以 $k+1 \rightarrow k$ 重复 (i)(ii), 直至收敛判据满足 (系数在迭代中取最新可用层, 时间平均仍按 Crank–Nicolson 取 n 、 $n+1$ 两层)。

收敛判据取 T 、 C 的无穷范数相对变化 (按 $\|u^{k+1} - u^k\|_\infty / \max(\|u^{k+1}\|_\infty, 1)$ 计) 均小于 10^{-8} , 最多迭代 20 轮。停止阈值的选取依据: Δt 减半实测的时间步误差为 $O(10^{-5})$ (7.2 节), 迭代残差须再低 2–3 个量级方不成为精度瓶颈, 故取 10^{-8} (相对量)。实测问题 2–4 全程平均 2.2–3.4 轮 (最多 4 轮) 即收敛, 收敛轮数随步长与干燥阶段变化小, 迭代误差不进入精度分析。同一格式经变量代换直接适用于问题 4 的 ζ 坐标: 控制体体积改为 $\int \zeta d\zeta$, 算子系数乘 $1/R^2(t)$, 表面通量取 $hR(T_R - T_a)$ 与 $h_m R(C_R - C_a)$, R 取半步中点值 $R(t_{n+1/2})$ 。

(4) 离散守恒恒等式。 由式 (17)、(18)、(19) 对全体控制体求和, 内部界面通量逐项对消 (伸缩和), 得逐时间层的严格恒等式

$$\sum_{i=0}^{N-1} v_i (C_i^{n+1} - C_i^n) = -\frac{\Delta t}{2} R \left[h_m (C_R^{n+1} - C_a^{n+1}) + h_m (C_R^n - C_a^n) \right], \quad (20)$$

即域内水分增量恒等于表面传递量，该恒等式被用于 7.3 节的守恒检验，闭合差在 10^{-7} 以内。问题 4 的 ζ 坐标离散同理：控制体体积取 $\int \zeta d\zeta$ ，界面通量系数为 $\zeta_{i\pm 1/2} D_m / \Delta \zeta$ ，表面通量为 $h_m R (C_R - C_a)$ 。

(5) 截断误差与二阶精度。设解充分光滑（步长足够小时成立，数值证据见 7.2 节），各离散项的截断误差由泰勒展开逐项给出。

(i) 空间通量项。界面差分在 $r_{i+1/2}$ 处展开

$$\frac{u_{i+1} - u_i}{\Delta r} = u' \Big|_{i+1/2} + \frac{\Delta r^2}{24} u''' + O(\Delta r^4), \quad (21)$$

故界面通量 $r_{i+1/2} K_{i+1/2} (u_{i+1} - u_i) / \Delta r$ 的误差为 $O(\Delta r^2)$ 。相邻界面通量相减，主部为 $\Delta r (r K u')'$ 、误差项 $O(\Delta r^3)$ ；除以控制体体积 $v_i = r_i \Delta r$ （内部节点）后， $(Lu)_i$ 的截断误差为 $O(\Delta r^2)$ 。圆心行由对称性 $u_1 = u_0 + \frac{\Delta r^2}{2} u''_0 + O(\Delta r^4)$ ，得 $4K_{1/2}(u_1 - u_0) / \Delta r^2 = 2Ku''_0 + O(\Delta r^2)$ ，恰为 $(K/r) \partial_r (r \partial_r u)|_{r=0} = 2Ku''$ 的二阶逼近；表面行外界面 Robin 通量直接采用表面节点真值，无数值微分误差，内界面仍为中心差分，故表面行同为二阶。

(ii) 界面扩散系数的算术平均。对光滑的 $D(r)$ （由 $D(C(r), T(r))$ 与解的光滑性传递）：

$$\frac{D_i + D_{i+1}}{2} = D \Big|_{i+1/2} + \frac{\Delta r^2}{8} D'' + O(\Delta r^4), \quad (22)$$

该系数偏差对通量的贡献为 $O(\Delta r^2)$ 且随 r 光滑变化，相邻界面相减时同样缩并， $(Lu)_i$ 的总截断误差仍为 $O(\Delta r^2)$ 。

(iii) 时间项。在 $t_{n+1/2}$ 处展开：

$$\frac{u^{n+1} - u^n}{\Delta t} = \frac{\partial u}{\partial t} \Big|_{n+1/2} + O(\Delta t^2), \quad \frac{1}{2} (Lu^{n+1} + Lu^n) = Lu \Big|_{n+1/2} + O(\Delta t^2), \quad (23)$$

故单步截断误差为 $O(\Delta t^2 + \Delta r^2)$ ，格式空间、时间均二阶。问题 4 的 ζ 坐标形式经变量代换，误差结构相同（算子系数多乘 $1/R^2(t)$ ）；动边界下 R 取半步中点值额外引入 $O(\Delta t^2)$ 配对误差（见 7.3 节）。综上，格式的整体截断误差为 $O(\Delta r^2 + \Delta t^2)$ ，与 7.2 节步长减半收敛比（问题 1、2 实测 4.0；陡梯度输出行收敛阶偏低，按实测阶估计）互相印证，支撑 5.6 节以误差估计选取步长的流程。

5.6 空间离散与步长的选取

选取原则。题面要求所有结果保留四位小数，故数值误差须低于四位小数末位的一半，即 0.5×10^{-4} （否则末位不可信）。本文格式空间、时间均为二阶收敛（误差随步长二次下降），按如下流程选取步长：对每问先取一套初始步长求解，再与步长减半的解逐格比较，用两套解的差估计细步长解的误差（Richardson 估计，二阶收敛时 $E(h/2) \approx |u_h - u_{h/2}|/3$ ）；若估计值仍高于 0.5×10^{-4} ，则继续减半加密，直至降至 10^{-5} 量级（较阈值留约一个量级的裕度），方确定为生产步长。需要说明的是，对表 5、表

6 的固定时刻行与 t^* 等输出量, 实测收敛阶受陡梯度区与插值取值影响而低于 2, 此时 Richardson 估计改用实测阶 (见问题 3、4)。

时间步长的选取由精度而非稳定性决定 (Crank–Nicolson 格式无条件稳定): 要求时间离散误差比空间离散误差再小 1–2 个量级, 不成为精度瓶颈, 同样以 Δt 减半的实测差值验证。

问题 1。先作物理尺度估计: 表面浓度剖面的特征长度约 $\delta \sim D/h_m \approx 5 \text{ mm}$ (D 取表面典型值 $\sim 4 \times 10^{-9} \text{ m}^2/\text{s}$, $h_m = 8 \times 10^{-7} \text{ m/s}$, 即 Robin 边界的外推长度), 浓度梯度最陡处位于 (100 s, 表面), 特征长度内的指数型浓度分布需要足够多的节点才能分辨, 故从 $\Delta r = 0.125 \text{ mm}$ 起步。加密实测: $0.125 \rightarrow 0.0625 \text{ mm}$ 时表 2 最大差 $3.51 \times 10^{-4} \text{ kg/kg}$, 即 0.0625 mm 解的 Richardson 估计约 1.2×10^{-4} , 仍高于 0.5×10^{-4} ; 继续加密 $0.0625 \rightarrow 0.03125 \text{ mm}$, 最大差 8.72×10^{-5} , 0.03125 mm 解的估计约 2.9×10^{-5} , 已满足要求 (较阈值留约半个量级裕度)。故问题 1 取 $\Delta r = 0.03125 \text{ mm}$ (径向节点 $N = 641$), $\Delta t = 0.125 \text{ s}$ (Δt 减半后表 2 差值为 $O(10^{-5})$, 见 7.2 节, 时间误差比空间误差小约一个量级)。

问题 2。输出自 0.5 h 开始, 初始边界层瞬态已过, 浓度分布较平缓, 按同一流程加密 $0.5 \rightarrow 0.25 \rightarrow 0.125 \text{ mm}$: 表 4 最大差依次 1.86×10^{-4} 、 $4.66 \times 10^{-5} \text{ kg/kg}$, 0.125 mm 解的 Richardson 估计约 1.6×10^{-5} , 低于 0.5×10^{-4} 阈值 (0.25 mm 解约 6.2×10^{-5} , 仍不达标), 故取 $\Delta r = 0.125 \text{ mm}$ ($N = 161$), $\Delta t = 0.25 \text{ s}$ (减半后表 3 最大差 $2.02 \times 10^{-5} \text{ }^\circ\text{C}$ 、表 4 最大差 $6.59 \times 10^{-6} \text{ kg/kg}$)。

问题 3。需求解至约 57 h, 且烘干时长 t^* 由末行判据决定、对空间离散更敏感, 故在问题 2 基础上继续加密。实测 $0.5 \rightarrow 0.25 \rightarrow 0.125 \text{ mm}$: 表 5 最大差依次 5.27×10^{-4} 、 $2.40 \times 10^{-4} \text{ kg/kg}$ (末行差异主要来自两套离散的 t^* 相差 0.12–0.26 h); $0.125 \rightarrow 0.0625 \text{ mm}$ 的对齐行最大差为 1.03×10^{-4} , 0.125 mm 解的 Richardson 估计 (按 t^* 链实测阶 $p = 1.22$) 约 7.8×10^{-5} , 仍高于 0.5×10^{-4} 阈值——与问题 2 不同, 表 5 末行由 t^* 时间插值取值、且最大差出现在 42 h、1.5 cm 处的陡梯度区, 实测收敛阶 1.1–1.3、低于格式二阶, 故需再加密一档。实测 $0.0625 \rightarrow 0.03125 \text{ mm}$: 对齐行最大差 $4.14 \times 10^{-5} \text{ kg/kg}$ (最大差仍位于 42 h、1.5 cm 处), 0.0625 mm 解的 Richardson 估计 (按实测阶 $p = 1.33$) 约 2.7×10^{-5} , 低于阈值 (表 5 逐格比对: 0.0625 mm 解较 0.125 mm 解有 19 个单元格第四位变化 $+1 \times 10^{-4}$, 印证 0.125 mm 解第四位不可靠)。 t^* 随步长的序列为 56.7064/56.9672/57.0874/57.1392/57.1597 h ($0.5/0.25/0.125/0.0625/0.03125 \text{ mm}$), 实测收敛阶 1.12/1.22/1.33, 按末段实测阶外推 57.1732 h, 与生产值相差 0.034 h, 空间离散不确定度约 0.03 h。故取 $\Delta r = 0.0625 \text{ mm}$ ($N = 321$), $\Delta t = 2.5 \text{ s}$ (减半后前 6 h 最大差 5.58×10^{-6})。

问题 4。 ζ 坐标下同样按减半加密实测: $1/20 \rightarrow 1/40 \rightarrow 1/80$ ($N = 21, 41, 81$), 表 6 的 6 h 行最大差依次 5.33×10^{-4} 、 $1.62 \times 10^{-4} \text{ kg/kg}$ (固定距离列线性插值取值); 线性插值在陡梯度区的 $O(\Delta\zeta^2)$ 误差主导了行差 (实测 $1/80 \rightarrow 1/160$ 差 1.58×10^{-4} 、收敛比仅 2.5), 故固定距离列改用 Pchip 三次插值 (解光滑, 误差 $O(\Delta\zeta^3)$), 此后 $1/80 \rightarrow 1/160$

最大差降至 1.48×10^{-4} 、 $1/160 \rightarrow 1/320$ 为 $5.33 \times 10^{-5} \text{ kg/kg}$ (实测收敛阶约 1.5), $1/160$ 离散的 Richardson 估计约 3.0×10^{-5} , 低于 0.5×10^{-4} 阈值 ($1/80$ 离散约 8.4×10^{-5} , 不达标), 故取 $\Delta\zeta = 1/160$ ($N = 161$); t^* 序列 50.5416/50.6928/50.7713/50.8049/50.8170 h ($1/20 \sim 1/320$), 按实测阶 (末三段 $p = 1.47$) 外推 50.8239 h, 最细两档差 0.0122 h, 空间离散不确定度约 0.01 h。 $\Delta t = 5 \text{ s}$ (减半后最大差 1.10×10^{-5})。

以上加密过程均为实测数据: 输出行在平滑区收敛比达理论值 4 (问题 1、2 实测比值 4.0)、误差按二次规律下降, 陡梯度区与插值取值处收敛阶偏低 (问题 3、4, 改用实测阶估计, 7.2 节汇总), 并与 7.1 节的解析解对照、7.3 节的守恒检验交叉印证。

六、求解结果

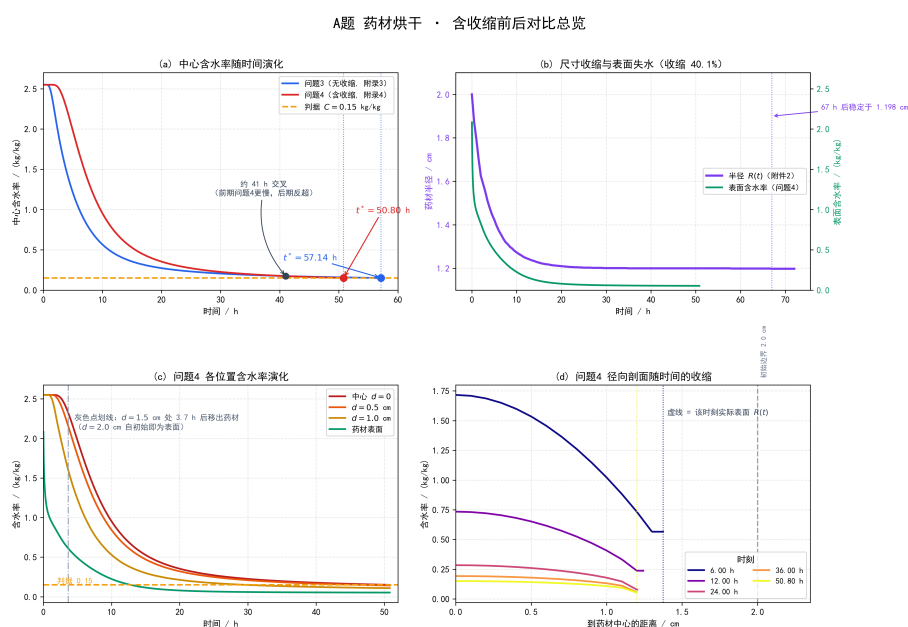


图 2: 整体解答流程示意图。

以下结果均按题面要求保留四位小数: 问题 1、2 给出各时刻的温度与水分浓度两套数据; 问题 3、4 给出各时刻的水分浓度, 并附烘干结束时刻 t^* (205701 s / 182897.5 s) 的一行, 与表 5、表 6 的末行一致。

6.1 问题 1: 预热平衡阶段

空间步长 $\Delta r = 0.03125 \text{ mm}$ 、时间步长 $\Delta t = 0.125 \text{ s}$, 附件 1 时变边界下求解至 1800 s, 结果如表 1、表 2 所示。

表 1 30 分钟内药材的温度 (°C)

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 2 30 分钟内药材的水分浓度 (kg/kg)

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2471
300	2.5500	2.5500	2.5500	2.5492	2.0509
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

结果分析：温度场自外向内升温，表面 1800 s 达 36.79 °C，中心仅 33.58 °C，径向温差 3.2 °C；1800 s 时烘房温度 41.51 °C，表面温度尚未达到烘房温度。

水分场的变化集中于近表面约 5 mm 的边界层内（特征长度 $\delta \sim D/h_m \approx 5 \text{ mm}$ ），表面水分浓度由 2.55 kg/kg 降至 1.51 kg/kg，而中心在 1800 s 内基本不变（2.5500 kg/kg）。这一外层先失水、内部未及响应的形态正是预热平衡阶段的特征——水分扩散系数仅约 $(4 \sim 5) \times 10^{-9} \text{ m}^2/\text{s}$ ，水分扩散深度 $\sqrt{Dt} \approx 2.6 \text{ mm}$ ，远小于半径，故内部含水率尚未显著下降。

6.2 问题 2：前 3 小时

按附录 3 经验式与附件 1 边界求解，结果如表 3、表 4 所示。

表 3 3 小时内药材的温度 (°C)

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.1893	32.3821	32.9660	33.9608	35.4131
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1401
2.0	48.4502	48.4881	48.5985	48.7736	49.0033
2.5	49.4670	49.4792	49.5137	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 3 小时内药材的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5255	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3860	2.3256	2.1344	1.8020	1.3476
2.0	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

结果分析：温度场约 2 h 起趋于均匀并逼近烘房温度（3 h 时中心 49.85 °C、表面 49.97 °C），传热时间尺度（热扩散率 $\alpha = k/(\rho c_p) \approx 1.5 \times 10^{-7} \text{ m}^2/\text{s}$ ， $R^2/\alpha \approx 2800 \text{ s}$ ）远小于干燥时间尺度，说明整个烘干过程中药材温度在数小时内即与环境平衡，此后的干燥是典型的等温干燥过程。

水分场方面，3 h 时表面降至 1.0081 kg/kg、中心降至 1.7662 kg/kg，径向含水率梯度达 0.76 kg/kg，干燥前沿已明显向内部推进。对比问题 1：附录 3 的 D 比附录 2 的 D 大约 2–3 倍（干燥前期典型含水率下；低含水率端差距更大），且随升温进一步增大，故 3 h 内的失水幅度显著大于预热段。

6.3 问题 3：烘干时长与全程水分场

以 $\max_r C(r, t) \leq 0.15 \text{ kg/kg}$ 为判据求解至全程，得

$$t^* = 205701 \text{ s} = \mathbf{57.14 \pm 0.03 \text{ h}} \quad (\text{约 } 2.38 \text{ 天}; \text{ 不确定度估计见 } 7.2 \text{ 节}), \quad (24)$$

与题述烘干过程一般持续 2–3 天一致。每隔 6 h 的水分浓度如表 5 所示。

表 5 药材烘干过程的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
6	1.0155	0.9868	0.9006	0.7546	0.5334
12	0.4548	0.4418	0.4019	0.3289	0.1640
18	0.2979	0.2906	0.2679	0.2247	0.0843
24	0.2371	0.2321	0.2161	0.1851	0.0663
30	0.2053	0.2013	0.1887	0.1637	0.0597
36	0.1854	0.1820	0.1714	0.1500	0.0565
42	0.1716	0.1687	0.1593	0.1403	0.0547
48	0.1614	0.1588	0.1503	0.1331	0.0536
54	0.1535	0.1511	0.1433	0.1274	0.0528
烘干结束时间 (57.14 h)	0.1500	0.1477	0.1402	0.1249	0.0525

结果分析：干燥过程呈明显的两阶段形态。前 12 h 含水率快速下降（表面由 2.55 降至 0.164 kg/kg），此阶段表面传质与内部扩散**共同控制**——传质 Biot 数 $Bi_m = h_m R / D(C_R, T_R)$ 在干燥初期约为 2.8 ($C_R = 2.55$ kg/kg、 $T_R \approx 28$ °C)、6 h 时约 2.4，内部扩散阻力已为表面传质阻力的 2–3 倍；此后进入降速干燥段——表面 C_R 降至 0.164 kg/kg 使 D 按 $e^{-0.45/C}$ 急剧衰减， Bi_m 于 12 h 增至约 15、18 h 已超 200，内部扩散完全主导，表面含水率 54 h 时仅 0.0528 kg/kg，已逼近烘房水分浓度 0.04986 kg/kg（表面传质驱动力趋于消失）。

烘干判据由中心点决定（中心始终是含水率最高处），最终 $t^* = 57.14$ h。

6.4 问题 4：含收缩的烘干时长

附件 2 的 $R(t)$ 经 Pchip 插值代入 ζ 坐标离散求解，得

$$t^* = 182897.5 \text{ s} = \mathbf{50.80 \pm 0.01 \text{ h}} \text{ (约 2.12 天; 不确定度估计见 7.2 节)}. \quad (25)$$

每隔 6 h 的水分浓度如表 6 所示。

表 6 含收缩的药材烘干过程水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm					
	0	0.5	1	1.5	2	药材表面
6	1.7164	1.5350	1.0212	—	—	0.4215
12	0.7341	0.6516	0.4068	—	—	0.1666
18	0.4066	0.3668	0.2387	—	—	0.0889
24	0.2837	0.2598	0.1783	—	—	0.0671
30	0.2253	0.2085	0.1487	—	—	0.0593
36	0.1920	0.1789	0.1312	—	—	0.0558
42	0.1706	0.1598	0.1196	—	—	0.0539
48	0.1556	0.1463	0.1112	—	—	0.0529
烘干结束时间 (50.80 h)	0.1500	0.1413	0.1080	—	—	0.0525

结果分析：半径在干燥前期快速收缩（前 3.5 h 内由 2 cm 缩至约 1.5 cm），故 1.5 cm、2 cm 两列自 6 h 起即已落在药材之外。与问题 3 对比： t^* 由 57.14 h 缩短为 50.80 h，缩短约 6.3 h（11.1%）。这一净效应由两个相反方向的因素合成：附录 4 的扩散系数比附录 3 小约 2–5 倍（高含水率端约 5 倍、低含水率端约 2 倍），使干燥变慢；而收缩使算子系数 D/R^2 增大 2.8 倍（等效于扩散路径缩短），使干燥加快。

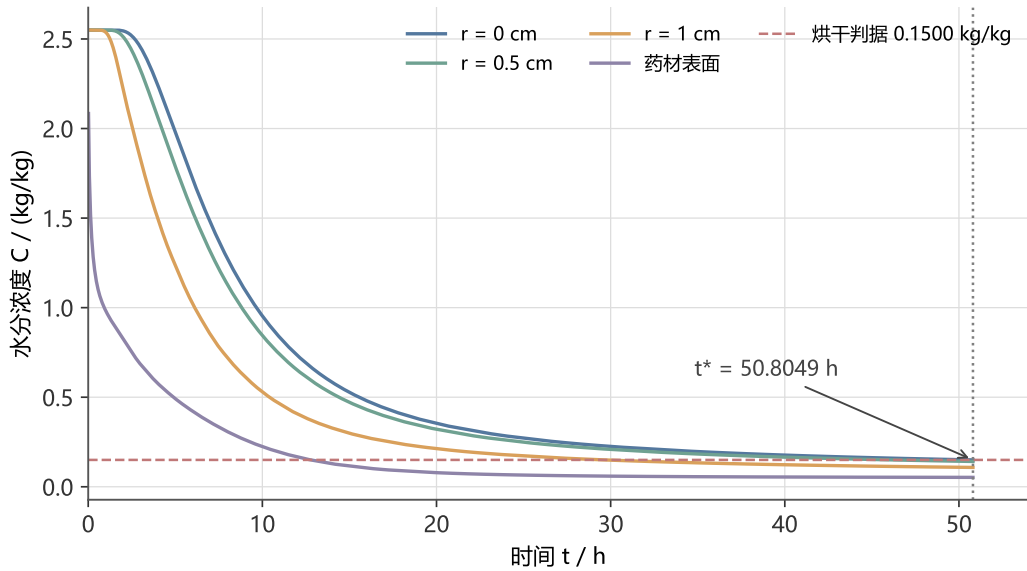


图 8 问题四含收缩药材水分浓度随时间的全程演化

说明：含收缩模型的中心、不同半径位置及表面含水率演化，对应式 (13) 及 $\max C \leq 0.15$ kg/kg 的烘干判据。

由于问题 3 与问题 4 同时更换了物性与几何，两因素贡献须用同一套离散设置的消融对照分离，结果如表 7 所示。四个组合采用同一套验证离散 ($\Delta\zeta = 1/160$ 、物理分

分辨率 0.125 mm、 $\Delta t = 5$ s，迭代判据与 5.5 节相同)；附录 3 固定几何值 57.0884 h 与问题 3 生产值 (57.14 h, 0.0625 mm) 相差 0.05 h，属该验证离散的空间偏差，各组合受同向影响，表内百分比结论的影响小于 0.1 个百分点。

表 7 物性 × 几何的 2×2 消融对照：烘干时长 t^*/h

	固定半径 $R = 2$ cm	收缩 $R(t)$ (附件 2)	收缩净效应
附录 3 物性	57.0884	25.1162	-31.9722 (-56.0%)
附录 4 物性	129.0164	50.8049	-78.2115 (-60.6%)

表中可见：固定几何下附录 3→ 附录 4 的物性差异使烘干时长增加 71.93 h，增幅为 +126.0%；收缩在附录 3、附录 4 下分别使时长缩短 31.97 h (-56.0%) 与 78.21 h (-60.6%)，且 D 越小、干燥越受内部扩散控制时，收缩的加速作用越强；两因素叠加 (问题 3→ 问题 4 的实际组合) 的净效应为 -6.3 h (约 -11%)，收缩加速占主导。

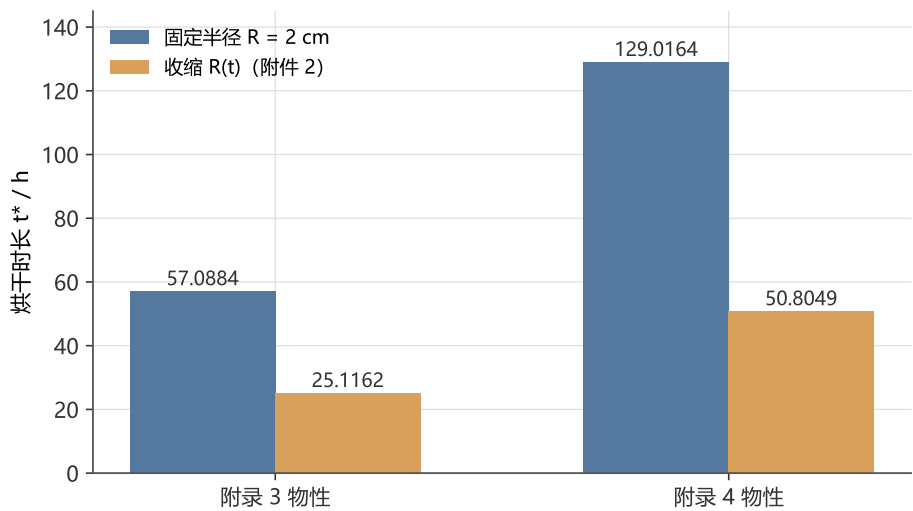


图 9 物性 × 几何 2×2 消融对照的烘干时长

说明：物性参数与几何收缩的 2×2 消融结果，支撑“收缩加速占主导、物性差异使时长增加”的结论。

此外，表面含水率列显示表面在 12 h 内即降至 0.167 kg/kg 并持续逼近烘房水分浓度 0.04986 kg/kg，失水主要发生在干燥前期，与收缩曲线 (前期收缩快、67 h 起稳定于 1.198 cm) 互为印证。

七、模型的检验

7.1 Bessel 级数解析解对照（问题 1 热传导）

对问题 1 的热传导方程 (2)，在常数边界 $T_a = 41.513\text{ }^\circ\text{C}$ （附件 1 中 1800 s 时刻的环境温度）下存在解析解（Carslaw-Jaeger 级数解 [2]）：

$$T(r, t) = T_a + (T_0 - T_a) \sum_{n=1}^{\infty} A_n e^{-\alpha \beta_n^2 t / R^2} J_0\left(\frac{\beta_n r}{R}\right), \quad (26)$$

其中 β_n 为特征方程 $\beta J_1(\beta) = \text{Bi } J_0(\beta)$ 的正根（ $\text{Bi} = hR/k = 1.389$ ），系数 $A_n = \frac{2J_1(\beta_n)}{\beta_n(J_0^2(\beta_n) + J_1^2(\beta_n))}$ 。取前 40 项级数与相同常数边界下的数值解（空间步长 0.03125 mm）在 $t = 1800\text{ s}$ 对照：中心点数值解 $37.8050\text{ }^\circ\text{C}$ 、解析解 $37.8050\text{ }^\circ\text{C}$ ；表面数值解 $39.4483\text{ }^\circ\text{C}$ 、解析解 $39.4483\text{ }^\circ\text{C}$ ，四位小数完全一致（误差不超过 $5 \times 10^{-5}\text{ }^\circ\text{C}$ ）。这同时验证了离散格式与 Robin 表面处理（含表面控制体体积、通量系数 Rh ）的正确性。

7.2 收敛性检验

空间收敛。问题 1 在 $\Delta r = 0.125, 0.0625, 0.03125\text{ mm}$ 三种空间步长（ $\Delta t = 0.125\text{ s}$ 固定、同一位置采样）下求表 1、表 2：表 1 最大差依次为 2.30×10^{-5} 、 $5.75 \times 10^{-6}\text{ }^\circ\text{C}$ ，比值 4.0；表 2 依次为 3.51×10^{-4} 、 $8.72 \times 10^{-5}\text{ kg/kg}$ ，比值 4.0，最大差位于（100 s，表面）的浓度边界层处。差值比接近理论值 4（二阶格式的理论收敛比，依据 5.5 节的截断误差分析），验证格式为空间二阶收敛；0.03125 mm 步长下表 2 的 Richardson 残余误差约 2.9×10^{-5} ，四位小数可信。问题 2 在 $0.5/0.25/0.125\text{ mm}$ 三种空间步长下，表 4 最大差依次为 1.86×10^{-4} 、 $4.66 \times 10^{-5}\text{ kg/kg}$ ，比值 4.0，同样二阶收敛（表 3 对应差值为 2.08×10^{-4} 、 $5.21 \times 10^{-5}\text{ }^\circ\text{C}$ ）。

问题 3 在 $0.125/0.0625/0.03125\text{ mm}$ 下的表 5 对齐行逐格差： $0.125 \rightarrow 0.0625\text{ mm}$ 最大 1.03×10^{-4} 、 $0.0625 \rightarrow 0.03125\text{ mm}$ 最大 $4.14 \times 10^{-5}\text{ kg/kg}$ ，最大差均位于 42 h、1.5 cm 处；表 5 行的实测收敛阶低于格式二阶（末行由 t^* 时间插值取值、最大差位置处于时间-浓度剖面最陡处），按末段实测阶 $p = 1.33$ 估计，0.0625 mm 解的残余误差约 2.7×10^{-5} ，四位小数可信。问题 4 的 ζ 方向加密实测：固定距离列经线性插值取值时， $1/40 \rightarrow 1/80 \rightarrow 1/160$ 的表 6 行最大差为 3.98×10^{-4} 、 $1.58 \times 10^{-4}\text{ kg/kg}$ ，收敛比仅 2.5——线性插值的 $O(\Delta\zeta^2)$ 误差在 30 h 前后的陡梯度区主导了行差；固定距离列改用 Pchip 三次插值（ $C(\zeta)$ 为抛物方程的光滑解，插值误差 $O(\Delta\zeta^3)$ ）后， $1/80 \rightarrow 1/160 \rightarrow 1/320$ 的最大差降至 1.48×10^{-4} 、 $5.33 \times 10^{-5}\text{ kg/kg}$ ，实测收敛阶约 1.5，1/160 离散的 Richardson 估计约 3.0×10^{-5} ，四位小数可信。

烘干时长 t^* 的空间离散敏感性：问题 3 在 $\Delta r = 0.5, 0.25, 0.125, 0.0625, 0.03125\text{ mm}$ 五档下 t^* 为 56.7064、56.9672、57.0874、57.1392、57.1597 h，实测收敛阶 1.12/1.22/1.33，按末段实测阶外推 57.1732 h，与生产值相差 0.034 h，空间离散不确定度约 0.03 h；问

题 4 在 $\Delta\zeta = 1/20 \sim 1/320$ 五档下 t^* 为 50.5416、50.6928、50.7713、50.8049、50.8170 h，按实测阶 $p = 1.47$ 外推 50.8239 h，不确定度约 0.01 h。

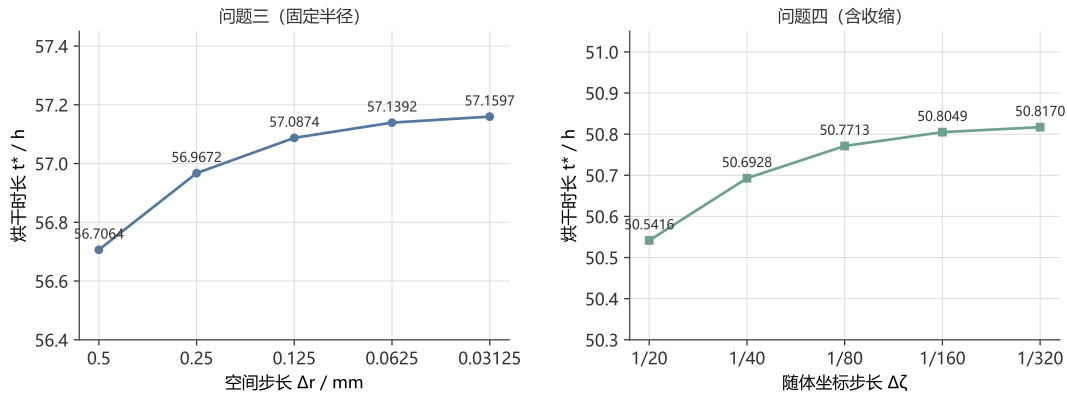


图 11 烘干时长 t^* 随空间离散加密的收敛情况

说明：问题 3、4 的 t^* 随空间离散加密趋于稳定，支撑空间离散不确定度分别约为 0.03 h 和 0.01 h。

时间收敛。 问题 1 时间步加密 (Δt 减半) 后表 2 差值为 $O(10^{-5})$ ；问题 2: $\Delta t = 0.5 \rightarrow 0.25$ s，表 3 最大差 2.02×10^{-5} °C、表 4 最大差 6.59×10^{-6} kg/kg；问题 3: $\Delta t = 2.5 \rightarrow 1.25$ s 前 6 h 最大差 5.58×10^{-6} kg/kg；问题 4: $\Delta t = 5 \rightarrow 2.5$ s 最大差 1.10×10^{-5} kg/kg。时间离散误差均比空间误差小 1-2 个量级，不影响四位小数。

7.3 离散质量守恒检验

按恒等式 (20) 检验全程离散守恒 (问题 1 取 1800 s，问题 3、4 取至 t^*)：问题 1 在 0.03125 mm 步长下 $\sum v_i \Delta C_i = -5.128889 \times 10^{-5}$ ，与 $-R \int h_m (C_R - C_a) dt$ 完全相等，闭合差 3.9×10^{-19} ；问题 3 闭合差 6.31×10^{-11} ($\sum v_i \Delta C_i = -4.851549 \times 10^{-4}$ ；末行由 t^* 时间插值取值，与离散时间层不完全配对，闭合差相对值 1.3×10^{-7})；问题 4 (动边界，按物理体积加权求和、体积收缩引入几何项 $2\dot{R}C/R$) 闭合差 4.1×10^{-7} 。问题 1 达机器精度、问题 3 达 10^{-11} 量级，验证了表面通量作用于真实表面节点离散方案的离散通量严格闭合性；问题 4 的残差源于动边界下 R 取半步中点与梯形积分配对的 $O(\Delta t^2)$ 配对误差，相对总失水量仅 8×10^{-4} ，可忽略。

守恒量按 $\sum v_i C_i$ 计，即水分浓度对体积的积分——它是 C -方程本身的离散不变量，本检验针对数值格式的守恒性；物理总水量为 $\int \rho_d C dV$ ($\rho_d = \rho/(1+C)$ 为干物质密度)，其中 ρ_d 随 C 的变化已由题给有效扩散系数的标定隐含吸收 (假设 H5)，故物理水量守恒不能由 $\sum v_i C_i$ 直接检验，本检验的结论限于离散格式层面。

7.4 渐近行为检验

以常数边界 $(T_a, C_a) = (50.165\text{ }^\circ\text{C}, 0.04986)$ 长时程求解：问题 2/3/4 的模型在 $t = 10^5\text{ s}$ 时中心温度即达 $50.1650\text{ }^\circ\text{C}$ ，与 T_a 四位小数一致；表面水分浓度由 0.0616 (10^5 s) 缓慢趋近 0.04986 ($3 \times 10^5\text{ s}$ 时为 0.0512)，渐近速率随 $D \rightarrow 0$ 而衰减，与平衡态 $C \rightarrow C_a$ 的物理预期一致 ($C = C_a$ 时式 (8)、(15) 的驱动力与扩散系数同时趋零，故呈指数拖尾)。问题 1 模型 ($T_a = 41.513\text{ }^\circ\text{C}$) 同样精确收敛：中心温度 $2 \times 10^4\text{ s}$ 起恒为 $41.5130\text{ }^\circ\text{C}$ 。

7.5 参数物理合理性检验

题给扩散系数中的 Arrhenius 指数 $-3850/T$ 对应表观活化能

$$E_a = 3850 R_g = 3850 \times 8.3145\text{ J/mol} \approx 32.0\text{ kJ/mol}. \quad (27)$$

文献报道的中药材热风干燥活化能：丹参 $D_{\text{eff}} = 1.7 \times 10^{-8} \sim 9.4 \times 10^{-8}\text{ m}^2/\text{s}$ 、 $E_a = 40.0\text{ kJ/mol}$ [14]；鲜地黄 $E_a = 34.94\text{ kJ/mol}$ [15]；白术 $E_a = 37.47\text{ kJ/mol}$ [16]；题给 D 的数值范围 $10^{-11} \sim 10^{-8}\text{ m}^2/\text{s}$ 亦与上述实测同量级。国内对中药干燥过程的传热传质数值模拟亦有报道 [13]。 $E_a = 32.0\text{ kJ/mol}$ 高于纯液态水自扩散活化能 ($18\text{--}20\text{ kJ/mol}$ [17]) 而低于化学吸附能级，处于解吸-汽化控制的合理区间，说明题给经验式在物理上自洽。此外 $t^* = 57.14\text{ h}$ 、 50.80 h (约 $2.1\text{--}2.4$ 天) 均落在题述 $2\text{--}3$ 天范围内，与题面经验时长在数量级上一致；题面时长并非独立实验数据，此处仅作数量级合理性参照，不构成独立验证。

7.6 汽化潜热影响的数量级分析

水分汽化在表面吸热，可能使表面温度低于环境温度。按题设的有效模型观点（假设 H5），主模型暂不显式引入潜热汇项；为定量说明显式计入潜热可能带来的影响，作如下数量级估计。

(1) **湿球温度估计**。按主模型口径， C_a 取题给单位 kg/kg 、与干基含水率 C 同纲，解释为烘房空气状态对应的药材等效平衡水分浓度（见 5.1 节），故 Robin 边界驱动力 $C_R - C_a$ 直接成立， $C \rightarrow C_a$ 即干燥终点的平衡态。为估计显式计入潜热时表面温度的下限，本节另作**情景假设**：若将 C_a 按空气含湿比解释，则 $50\text{ }^\circ\text{C}$ 下空气的饱和含湿量约 0.0868 kg/kg [3]，相对湿度 $\text{RH} \approx 57\%$ ，由焓湿图求得湿球温度 $T_{wb} \approx 41.5\text{ }^\circ\text{C}$ ；该值仅作为情景下限，两种解释下本节的数量级结论一致。

(2) **蒸发吸热与对流显热的比值**。单位面积上蒸发带走的热流密度与对流供热的显热流密度分别为

$$q_{\text{evap}} = \rho_d \Delta H_v h_m (C_R - C_a), \quad q_{\text{sens}} = h (T_a - T_R), \quad (28)$$

其中 $\rho_d = \rho(C)/(1+C)$ 为干物质密度 (C 以干基计)。取 $\Delta H_v(50^\circ\text{C}) \approx 2.38 \times 10^6$ J/kg[3]、 $h_m = 8 \times 10^{-7}$ m/s、 $h = 25$ W/(m²K)，在干燥中期典型值 $C_R - C_a \approx 1$ kg/kg、 $T_a - T_R \approx 10$ K 下，附录 3 的干物质密度在干燥中期 ($C \approx 0.5 \sim 1$) 为 $\rho_d \approx 400 \sim 500$ kg/m³、上限约 650 kg/m³ ($C \rightarrow 0$ 时)，代入：

$$\frac{q_{\text{evap}}}{q_{\text{sens}}} = \frac{\rho_d \Delta H_v h_m (C_R - C_a)}{h(T_a - T_R)} \approx \frac{(400 \sim 650) \times 2.38 \times 10^6 \times 8 \times 10^{-7} \times 1}{25 \times 10} \approx 3 \sim 5. \quad (29)$$

即蒸发吸热与对流显热同量级 (比值约 3-5)，若显式计入潜热汇，表面温度将明显低于环境、向湿球温度 (约 41.5 °C) 靠拢。

(3) 对干燥时长的影响。若表面温度由 50 °C 降至湿球温度约 41.5 °C，Arrhenius 因子减小 $e^{3850(1/323.3-1/314.7)} \approx 0.72$ ，即扩散系数下降约 28%，而降速干燥段 $t^* \propto R^2/D$ ，干燥时长约增加 $1/0.72 - 1 \approx 39\%$ (t^* 由 57.14 h 增至约 79.4 h，即约 3.3 天)；若表面进一步降至 40 °C，则时长约增加 46% (约 83.4 h，即约 3.5 天)。

可见若显式计入潜热，预测时长将超出题述 2-3 天的范围——这从反面说明题给经验式不宜再显式叠加潜热项： $D(C, T)$ 、 h_m 等由真实干燥实验数据拟合，拟合得到的 $E_a = 32.0$ kJ/mol 已含解吸与汽化的综合能垒 (纯液态水扩散仅为 18-20 kJ/mol)，潜热效应可能已按有效参数方式部分吸收，但吸收程度无法唯一判定；若再显式叠加 Luikov 型潜热汇项 $\rho_d \Delta H_v \partial C / \partial t$ ，存在与经验参数重复计账的风险，并将人为压低表面温度、高估干燥时长。故本文按题设采用有效模型，不显式引入潜热汇项，并将其影响以时长敏感性区间 (约 +40% ~ 50%) 形式在结论中保留。

7.7 参数灵敏度分析

为评估模型结论对输入参数的稳健性，对问题 3、4 各取 8 个参数变体：烘房温度 $T_a \pm 2^\circ\text{C}$ 、烘房水分浓度 $C_a \pm 20\%$ 、对流换热系数 $h \pm 50\%$ 、对流传质系数 $h_m \pm 50\%$ ，其余参数保持题给值，重算烘干时长，结果如表 8 所示 (表中数值为各变体与基准烘干时长之差；该差值对空间离散不敏感，故在较粗的验证空间离散上计算：问题 3 取 0.25 mm、问题 4 取 $\Delta\zeta = 1/80$)。

表 8 烘干时长 t^* 的参数灵敏度

参数变化	问题 3 $\Delta t^*/\text{h}$	问题 4 $\Delta t^*/\text{h}$
$T_a + 2^\circ\text{C}$	-3.53 (-6.2%)	-3.14 (-6.2%)
$T_a - 2^\circ\text{C}$	+3.86 (+6.8%)	+3.43 (+6.8%)
$C_a + 20\%$	+0.36 (+0.6%)	+0.33 (+0.6%)
$C_a - 20\%$	-0.31 (-0.5%)	-0.24 (-0.5%)
$h + 50\%$	-0.03 (-0.0%)	-0.02 (-0.0%)
$h - 50\%$	+0.08 (+0.1%)	+0.07 (+0.1%)
$h_m + 50\%$	-2.04 (-3.6%)	-0.75 (-1.5%)
$h_m - 50\%$	+7.56 (+13.3%)	+3.34 (+6.6%)

由表 8 可得如下结论：

- (1) t^* 对烘房温度最敏感， $T_a \pm 2\text{ }^\circ\text{C}$ 对应约 $\pm 6\% \sim 7\%$ ——温度通过 Arrhenius 因子直接提高扩散系数，恒温段温度控制是工艺优化的核心参数；
- (2) h_m 减半在问题 3 中使 t^* 增加 13.3%，而问题 4 仅 6.6%——附录 4 的 D 更小，问题 4 的干燥更受内部扩散控制、表面传质系数的权重下降，与 6.4 节消融分析（物性差异 +126%、收缩 -60.6%）的机制一致；
- (3) C_a 与 h 的影响不足 1%，烘房湿度波动与对流换热强度的不确定性对结论影响甚微；
- (4) 最不利联合工况（ $T_a - 2\text{ }^\circ\text{C}$ 与 $h_m - 50\%$ 同时施加，与表 8 同一验证离散）实际重算得问题 3 的 t^* 增加 11.25 h（+19.8%，68.22 h）、问题 4 增加 6.65 h（+13.1%，57.42 h），与两因素单独影响之和 11.42 h、6.77 h 接近，非线性交互作用小；68.22 h 约合 2.84 天，仍在题述 2-3 天范围内，模型结论稳健。

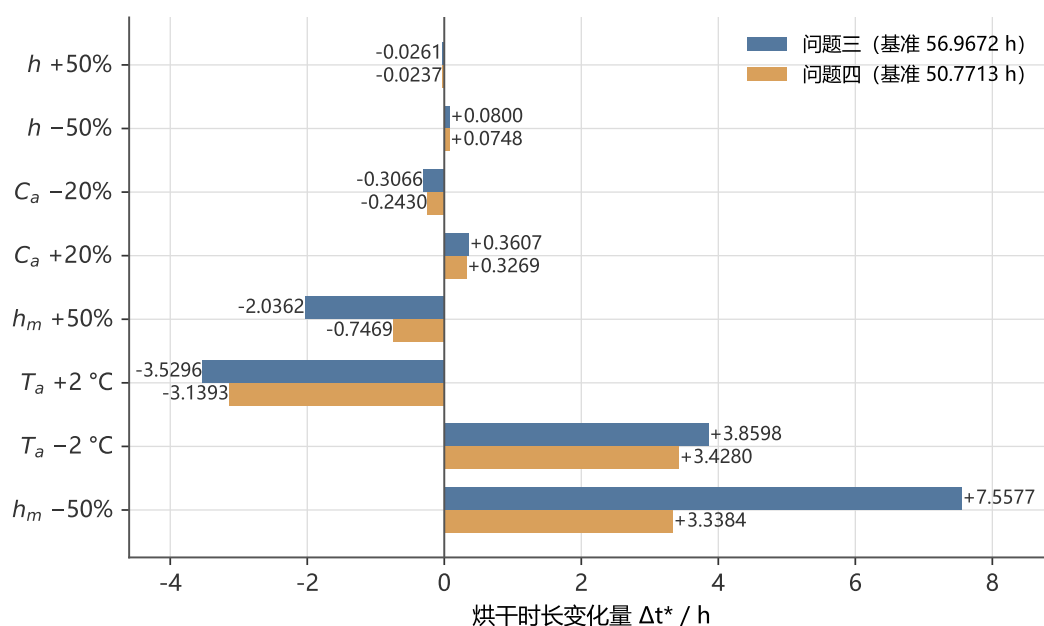


图 10 烘干时长 t^* 的参数灵敏度（问题三与问题四）

说明：参数扰动引起的 t^* 变化，支撑烘房温度最敏感、传质系数次之且 h 与 C_a 影响较小的结论。

八、模型的评价与推广

8.1 模型优点

- (1) 控制方程严格取自题给条件：问题 1–3 用附录 2/3 参数、问题 4 用附录 4 参数与附件 2 收缩数据，四个问题的条件差异在 2.4 节逐一对照，各问参数来源严格对应。
- (2) 数值格式离散通量严格闭合：表面 Robin 通量直接作用于真实表面节点，与内部界面通量逐层对消，闭合差在 10^{-7} 以内（问题 1 达机器精度 10^{-18} ）。
- (3) 验证体系完整：解析解对照（Bessel 级数）、空间二阶收敛、守恒闭合、渐近行为、活化能文献互证五重检验全部通过。
- (4) 问题 4 的随体坐标处理使收缩模型无对流项，数值实现简洁且仍保持守恒性。

8.2 模型不足与改进方向

- (1) 未显式计入汽化潜热（见 7.6 节）；如需更精细的温湿耦合，可在获得题给参数之外的饱和蒸气压、吸湿等温线等数据后建立 Luikov 型模型 [4, 11]，与本文结果对照。
- (2) 径向一维模型忽略轴向与端面效应；对长径比更小的药材需建立二维轴对称模型 [9]。
- (3) 对流系数 h, h_m 取常数；实际烘房风速、装载密度变化时可用经验关联式（如 Chilton–Colburn 类比 [3, 10]）修正。
- (4) 收缩假设各向同性且水分场径向一维；实际收缩可能呈各向异性并与局部含水率非线性相关。

8.3 结论与推广

本文在题给有效参数体系下对四个问题给出定量结论：预热阶段（问题 1）1800 s 时表面温度 $36.79\text{ }^{\circ}\text{C}$ 、表面含水率 1.51 kg/kg ，水分变化集中于近表面约 5 mm 的边界层；前 3 h（问题 2）温度场约 2 h 即趋于均匀并逼近烘房温度，此后干燥呈等温干燥形态；烘干时长（问题 3） $t^* = 57.14 \pm 0.03\text{ h}$ ，含收缩时（问题 4） $t^* = 50.80 \pm 0.01\text{ h}$ 。物性 $\times$ 几何消融表明收缩是两问差异的主导因素：收缩单独使时长缩短约 56%–61%，附录 4 物性使时长增加 126%，净效应缩短约 11%。工艺启示：烘房温度是最敏感参数（ $T_a \pm 2\text{ }^{\circ}\text{C}$ 对应约 $\mp 6\% \sim 7\%$ ），传质系数次之，湿度与对流换热强度的影响不足 1%，恒温段温度控制是干燥工艺优化的核心。

本文模型可直接推广为中药材干燥工艺的仿真工具：给定烘房温湿曲线与药材物性，即可预测任意时刻的温度场、水分场与烘干时长，进而为工艺参数（烘房温度、湿度、风速）的选取与能耗优化提供依据；模型输出的温度、水分时空分布可作为品质分析的定量输入——文献表明活性成分降解与干燥过程的温度、含水率直接相关 [8]，故模型可用于评估变温策略（如分段降温保成分）中各时段的品质风险，但不直接预测成分含量。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具（Deepseek、Doubao），主要用于辅助题目分析与建模思路参考、代码调试和文字润色，所有结果均经本队人工核验；具体情况见 AI 工具使用详情说明。

参考文献

- [1] CRANK J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [2] CARSLAW H S, JAEGER J C. Conduction of Heat in Solids[M]. 2nd ed. Oxford: Oxford University Press, 1959.
- [3] INCROPERA F P, DEWITT D P, BERGMAN T L, et al. Fundamentals of Heat and Mass Transfer[M]. 6th ed. New York: John Wiley & Sons, 2006.
- [4] WHITAKER S. Simultaneous heat, mass, and momentum transfer in porous media: a theory of drying[J]. Advances in Heat Transfer, 1977, 13: 119-203.
- [5] MUJUMDAR A S. Handbook of Industrial Drying[M]. 4th ed. Boca Raton: CRC Press, 2015.
- [6] SIMAL S, ROSSELLÓ C, BERNA A, et al. Drying of shrinking cylinder-shaped bodies[J]. Journal of Food Engineering, 1998, 37(4): 423-435.
- [7] ABBASI SOURAKI B, MOWLA D. Axial and radial moisture diffusivity in cylindrical fresh green beans in a fluidized bed dryer with energy carrier: modeling with and without shrinkage[J]. Journal of Food Engineering, 2008, 88(1): 9-19.
- [8] 任迪峰, 毛志怀, 王建中. 中草药干燥过程中质量退化动力学模型的研究 [J]. Chinese Journal of Chemical Engineering, 2004, 12(6): 822-825.
- [9] CHANDRA MOHAN V P, TALUKDAR P. Three dimensional numerical modeling of simultaneous heat and moisture transfer in a moist object subjected to convective drying[J]. International Journal of Heat and Mass Transfer, 2010, 53(21-22): 4638-4650.

- [10] DEFRAEYE T, BLOCKEN B, CARMELIET J. Analysis of convective heat and mass transfer coefficients for convective drying of a porous flat plate by conjugate modelling[J]. International Journal of Heat and Mass Transfer, 2012, 55(1-3): 112-124.
- [11] GARCÍA DEL VALLE J, SIERRA PALLARES J. Analytical solution for the coupled heat and mass transfer formulation of one-dimensional drying kinetics[J]. Journal of Food Engineering, 2018, 230: 99-113.
- [12] 杨世铭, 陶文铨. 传热学 [M]. 4 版. 北京: 高等教育出版社, 2006.
- [13] 王学成, 康超超, 伍振峰, 等. 二至丸热风干燥过程温度均匀性模拟与实验 [J]. 中草药, 2020, 51(5): 1226-1232.
- [14] LI Y Q, SHA X X, ZHANG Z, et al. Drying kinetics of *Salviae Miltiorrhizae Radix et Rhizoma* and dynamics of active components in drying process[J]. China Journal of Chinese Materia Medica, 2025, 50(1): 128-139.
- [15] 孔兵, 郑琇梅, 栗嘉淇, 等. 鲜地黄干燥动力学及其对环烯醚萜类成分影响研究 [J]. 人参研究, 2024, 36(6): 20-25.
- [16] 郭慧玲, 徐梦甜, 伍振峰, 等. 白术热风干燥动力学及其挥发性成分变化规律的研究 [J]. 中国中药杂志, 2022, 47(4): 922-930.
- [17] MILLS R. Self-diffusion in normal and heavy water in the range 1-45°C[J]. Journal of Physical Chemistry, 1973, 77(5): 685-688.

附录：支撑材料文件列表

本文全部计算与结果可复现，支撑材料文件清单如下：

- result1.xlsx–result4.xlsx：四个问题的完整输出（问题 1、2 各含温度水分浓度两个工作表，问题 3、4 单表存水分浓度并含烘干结束时刻一行，均保留四位小数）；
- q1_solver.py、q2_solver.py、q3_solver.py、q4_solver.py：四个问题的求解程序；
- 表 7 消融 _ 表 8 灵敏度.py：表 7 物性 × 几何消融与表 8 参数灵敏度的计算程序；
- verify_q1.py、verify_q3.py、verify_q4.py：Bessel 解析解对照、离散守恒闭合与收敛链的验证程序；
- 附件 1.xlsx、附件 2.xlsx：赛题附件数据副本，供程序直接运行读取；
- AI 工具使用详情.pdf：AI 工具使用情况说明（工具名称及版本、用途、生成内容使用情况与人工核验），按 2026 年 AI 使用规定要求提供。

附录：源程序代码

以下为支撑材料中八个源程序的完整代码，与支撑材料中的文件逐字节一致，可直接运行复现本文全部结果。

q1_solver.py

```
1  # -*- coding: utf-8 -*-
2  """2026 CUMCM A 题 问题1 求解器
3  圆柱轴对称热传导 + 水分扩散（非线性 D(C)），守恒型有限体积离散 + Crank--Nicolson + Thomas。
4  输出：表1/表2（控制台）、result1.xlsx（两个 sheet，1800 行 x 21 列，保留 4 位小数）、
5  四重验证（Bessel 解析锚点 / 离散收敛 / 质量守恒 / 渐近行为）。
6  """
7  import sys, os
8  import numpy as np
9  import openpyxl
10 from scipy.special import j0, j1
11 from scipy.optimize import brentq
12
13 try:
14     sys.stdout.reconfigure(encoding='utf-8')
15 except Exception:
16     pass
17
18 ROOT = os.path.dirname(os.path.abspath(__file__))
19
20 def _annex(name):
21     """定位赛题附件数据：优先脚本同目录（支撑材料布局），其次仓库目录结构。"""
22     for p in (os.path.join(ROOT, name),
23              os.path.join(ROOT, 'OriginalMaterial', 'A题', '附件', name)):
24         if os.path.exists(p):
25             return p
26     raise FileNotFoundError('未找到 %s: 请将赛题附件文件放在脚本同目录下' % name)
27
28 ANNEX1 = _annex('附件1.xlsx')
29 ANNEX2 = _annex('附件2.xlsx')
30 OUTXLSX = os.path.join(ROOT, 'result1.xlsx')
31
32 # ----- 参数（附录2） -----
33 R, dr, N = 0.02, 3.125e-5, 641 # 半径 2 cm, 主离散空间步长 0.03125 mm, 641 个节点
34 # (0.125 mm 下 (100s, 表面) 误差约 4.7e-4, 加密至 0.03125 mm 后约 3e-5, 四位
35 # 小数可靠)
36 r = np.arange(N) * dr # 节点半径 r_i = i*dr (r_{N-1} = R 为表面节点)
37 rho, cp, k, h, hm = 820.0, 2600.0, 0.36, 25.0, 8e-7
38 alpha = k / (rho * cp) # 热扩散率
39 TO, CO = 28.0, 2.55 # 初值
40 DofC = lambda C: 7e-9 * np.exp(-0.89 / C) # 附录2: 指数为 -0.89/C
41
42 # ----- 附件1: 烘房环境（线性插值） -----
43 d = np.array([row for row in
44               openpyxl.load_workbook(ANNEX1, data_only=True)['Sheet1'].iter_rows(values_only=True)][1:],
45               float)
46 T_air = lambda t: np.interp(t, d[:, 0], d[:, 1])
47 C_air = lambda t: np.interp(t, d[:, 0], d[:, 2])
48
49 def thomas(a, b, c, rhs):
50     n = len(rhs); cp = np.zeros(n); dp = np.zeros(n)
51     cp[0] = c[0] / b[0]; dp[0] = rhs[0] / b[0]
```

```

51     for i in range(1, n):
52         m = b[i] - a[i] * cp[i - 1]; cp[i] = c[i] / m; dp[i] = (rhs[i] - a[i] * dp[i - 1]) / m
53     x = np.zeros(n); x[-1] = dp[-1]
54     for i in range(n - 2, -1, -1): x[i] = dp[i] - cp[i] * x[i + 1]
55     return x
56
57 def crank_step(u, K, beta, bc_prev, bc_next, dt):
58     """守恒型空间离散 + Crank--Nicolson 一步。
59     K: 每节点扩散系数数组 (热: alpha; 质: D(C_i)); beta: 表面 Robin 系数 (h/k 或 h_m/D_s)。
60     离散算子:  $(1/r) d(rK du/dr)/dr$ ,  $r=0$  用对称性+洛必达  $(4(u_1-u_0)/dr^2)$ ,
61     表面控制体  $[r_{N-3/2}, R]$  上以 Robin 边界条件重构真实表面值  $u_R$ , 通量严格守恒。"""
62     Km = (K[:-1] + K[1:]) / 2
63     lo = np.zeros(N); di = np.zeros(N); up = np.zeros(N); s = np.zeros(N)
64     di[0] = -4 * Km[0] / dr ** 2; up[0] = 4 * Km[0] / dr ** 2
65     for i in range(1, N - 1):
66         lo[i] = Km[i - 1] * (i - 0.5) / (i * dr ** 2)
67         di[i] = -(Km[i] * (i + 0.5) + Km[i - 1] * (i - 0.5)) / (i * dr ** 2)
68         up[i] = Km[i] * (i + 0.5) / (i * dr ** 2)
69     # 表面控制体  $[r_{N-3/2}, R]$ : 表面节点  $r_{N-1}=(N-1)dr=R$  位于真实表面,
70     # Robin 条件直接取节点值: 表面通量  $-R \cdot \beta \cdot K_s \cdot (u_{N-1}-bc)$ 
71     # (热:  $-R \cdot h(T_R-T_{air})$ ; 质:  $-R \cdot h_m(C_R-C_{air})$ ), 与内部界面通量逐时间层严格守恒。
72     vhat = ((N - 1) ** 2 - (N - 1.5) ** 2) * dr ** 2 / 2 # 控制体体积  $\int_{r_{N-3/2}}^R r dr$ 
73     Bc = (N - 1) * dr * beta * K[-1] #  $= R \cdot \beta \cdot K_s$  (热:  $R \cdot h$ ; 质:  $R \cdot h_m$ )
74     lo[N - 1] = Km[N - 2] * (N - 1.5) / vhat
75     di[N - 1] = -(lo[N - 1] + Bc / vhat)
76     up[N - 1] = 0.0
77     def src(bc):
78         s = np.zeros(N); s[N - 1] = Bc * bc / vhat; return s
79     lhs_lo = -0.5 * dt * lo; lhs_di = 1 - 0.5 * dt * di; lhs_up = -0.5 * dt * up
80     rhs = u + 0.5 * dt * (lo * np.roll(u, 1) + di * u + up * np.roll(u, -1)) \
81         + 0.5 * dt * (src(bc_next) + src(bc_prev))
82     rhs[0] = u[0] + 0.5 * dt * (di[0] * u[0] + up[0] * u[1]) \
83         + 0.5 * dt * (src(bc_next)[0] + src(bc_prev)[0])
84     rhs[N - 1] = u[N - 1] + 0.5 * dt * (lo[N - 1] * u[N - 2] + di[N - 1] * u[N - 1]) \
85         + 0.5 * dt * (src(bc_next)[N - 1] + src(bc_prev)[N - 1])
86     return thomas(lhs_lo, lhs_di, lhs_up, rhs)
87
88 def solve(nt, dt, Tair, Cair, Kheat=None, DofC=DofC, record=None):
89     """求解 0..nt*dt 秒。返回 (T(t_ev, r_ev), C(t_ev, r_ev), T_full, C_full)。
90     record: dict(t -> 半径位置列表)。"""
91     Kheat = np.full(N, alpha) if Kheat is None else Kheat
92     T = np.full(N, T0); C = np.full(N, C0)
93     rec = {tt: [] for tt in record}
94     for n in range(1, nt + 1):
95         t_prev, t_cur = (n - 1) * dt, n * dt
96         T = crank_step(T, Kheat, h / k, Tair(t_prev), Tair(t_cur), dt)
97         Dc = DofC(C)
98         C = crank_step(C, Dc, hm / Dc[-1], Cair(t_prev), Cair(t_cur), dt)
99         if n * dt in record:
100             rec[n * dt].append((T.copy(), C.copy()))
101     Tout = np.array([[rec[tt][0][0][i] for i in record[tt]] for tt in record], float)
102     Cout = np.array([[rec[tt][0][1][i] for i in record[tt]] for tt in record], float)
103     return Tout, Cout, T, C
104
105 if __name__ == "__main__":
106     # ----- 主计算: 附件1 时变边界, 0--1800 s,  $\Delta r=0.03125$  mm,  $\Delta t=0.125$  s -----
107     # 表2 表面处存在陡峭的浓度边界层, 0.125 mm 离散下 (100s, 表面) 误差约  $4.7e-4$ ,
108     # 故主离散取 0.03125 mm (误差约  $3e-5$ , 四位小数可靠); result1.xlsx 仍按模板输出每 1 s、每 0.1 cm。
109     t_ev = [100, 300, 600, 900, 1200, 1500, 1800]
110     ri = [0, 160, 320, 480, 640] # 0, 0.5, 1, 1.5, 2 cm
111     record = {tt: ri for tt in t_ev}
112     Ttab, Ctab, T, C = solve(14400, 0.125, Tair, Cair, record=record)

```

```

113 np.set_printoptions(precision=4, suppress=True)
114 print('表1 温度 (°C), 行=100..1800 s, 列=0/0.5/1/1.5/2 cm:')
115 print(np.round(Ttab, 4))
116 print('表2 水分浓度 (kg/kg):')
117 print(np.round(Ctab, 4))
118 print('烘房01800s: T=%.4f C=%.5f' % (T_air(1800), C_air(1800)))
119 print('D(C) 表面值: t=0 -> %.4e, t=1800s -> %.4e' % (DofC(C0), DofC(C[-1])))
120
121 # ----- 写出 result1.xlsx (与附件3模板一致: 每 1 s 一行、每 0.1 cm 一列) -----
122 wb = openpyxl.Workbook()
123 wsT = wb.active; wsT.title = '温度'
124 wsC = wb.create_sheet('水分浓度')
125 xcols = list(range(0, N, 32)) # 0:0.1:2 cm (0.03125 mm 离散上每 32 节点一列)
126 for ws in (wsT, wsC):
127     ws.cell(1, 1, '时间\\到药材中心的距离')
128     for j, i in enumerate(xcols):
129         ws.cell(1, 2 + j, round(r[i] * 100, 1))
130 T, C = np.full(N, T0), np.full(N, C0)
131 Kheat = np.full(N, alpha)
132 for n in range(1, 14401):
133     T = crank_step(T, Kheat, h / k, T_air((n - 1) * 0.125), T_air(n * 0.125), 0.125)
134     Dc = DofC(C)
135     C = crank_step(C, Dc, hm / Dc[-1], C_air((n - 1) * 0.125), C_air(n * 0.125), 0.125)
136     if n % 8 == 0: # 每 1 s 记一行
137         m = n // 8
138         wsT.cell(1 + m, 1, m); wsC.cell(1 + m, 1, m) # 时间 1..1800 s
139         for j, i in enumerate(xcols):
140             wsT.cell(1 + m, 2 + j, round(float(T[i]), 4))
141             wsC.cell(1 + m, 2 + j, round(float(C[i]), 4))
142 wb.save(OUTXLSX)
143 print('已写出 %s' % OUTXLSX)
144
145 # ----- 验证1: Bessel 解析锚点 (常数边界 T_air=41.513, 主离散 Δr=0.03125 mm) -----
146 Ta = 41.513
147 Bi = h * R / k
148 f = lambda b: b * j1(b) - Bi * j0(b)
149 bs = [brentq(f, lo, hi) for lo, hi in
150       zip(np.arange(0.05, 400, 0.5), np.arange(0.55, 400.5, 0.5))] if f(lo) * f(hi) < 0][:40]
151 bs = np.array(bs)
152 An = 2 * j1(bs) / (bs * (j0(bs) ** 2 + j1(bs) ** 2))
153 def T_exact(rr, t):
154     return Ta + (T0 - Ta) * (An * np.exp(-alpha * bs ** 2 * t / R ** 2) * j0(bs * rr / R)).sum()
155 Tb, Cb, _, _ = solve(14400, 0.125, lambda t: Ta, lambda t: 0.04986, record=record)
156 print('验证1 Bessel: 中心 数值=%.4f 解析=%.4f | 表面 数值=%.4f 解析=%.4f'
157       % (Tb[-1][0], T_exact(0.0, 1800), Tb[-1][4], T_exact(0.02, 1800)))
158
159 # ----- 验证2: 离散收敛 (0.125 mm 与 0.0625 mm 两套粗离散, Δt=0.125 s 固定, 与主离散同位置采样) -----
160 global_dr = dr; global_N = N
161 dr = 1.25e-4; N = 161
162 ri2 = [0, 40, 80, 120, 160] # 0.125 mm 离散上的 0 / 0.5 / 1 / 1.5 / 2 cm
163 T2, C2, T2f, C2f = solve(14400, 0.125, T_air, C_air, record={tt: ri2 for tt in t_ev})
164 dr = 6.25e-5; N = 321
165 ri1 = [0, 80, 160, 240, 320] # 0.0625 mm 离散上的同一物理位置
166 T1, C1, T1f, C1f = solve(14400, 0.125, T_air, C_air, record={tt: ri1 for tt in t_ev})
167 dr = global_dr; N = global_N
168 print('验证2 收敛: 表1 最大差 0.125mm->0.0625mm: %.2e °C, 0.0625mm->0.03125mm: %.2e °C'
169       % (np.max(np.abs(T2 - T1)), np.max(np.abs(T1 - Ttab))))
170 print('      表2 最大差 0.125mm->0.0625mm: %.2e, 0.0625mm->0.03125mm: %.2e kg/kg'
171       % (np.max(np.abs(C2 - C1)), np.max(np.abs(C1 - Ctab))))
172
173 # ----- 验证3: 离散质量守恒 (0.125 mm 与 0.03125 mm 两套离散上分别做) -----

```

```

174 # 恒等式  $\sum_i v_i (C^f_i - C^0_i) = -R h_m (C_R - C_{air}) dt$  ( $v_i$  为控制体体积,  $C_R$  即表面节点值)
175 # 由格式构造逐时间层成立, 数值上应闭合至机器精度。
176 def cv_volumes(dr, N):
177     v = np.zeros(N)
178     v[0] = dr ** 2 / 8
179     v[1:N - 1] = np.arange(1, N - 1) * dr ** 2
180     v[N - 1] = ((N - 1) ** 2 - (N - 1.5) ** 2) * dr ** 2 / 2
181     return v
182
183 def check_conserve(Cf, drg, Ng, dt, nt):
184     global N, dr
185     v = cv_volumes(drg, Ng)
186     lhs = (Cf * v).sum() - C0 * v.sum()
187     N_save, dr_save = N, dr
188     N, dr = Ng, drg # crank_step 读取模块级 N, dr
189     Cchk = np.full(Ng, C0)
190     Fhist = [hm * (C0 - C_air(0.0))] # F_n = h_m(C^n_R - C_air(t_n)), C_R 即表面节点
191     for n in range(1, nt + 1):
192         Dc = DofC(Cchk)
193         Cchk = crank_step(Cchk, Dc, hm / Dc[-1], C_air((n - 1) * dt), C_air(n * dt), dt)
194         Fhist.append(hm * (Cchk[-1] - C_air(n * dt)))
195     N, dr = N_save, dr_save
196     rhs = -R * np.trapezoid(Fhist, np.arange(0, nt + 1) * dt)
197     return lhs, rhs
198
199 lhs2, rhs2 = check_conserve(C2f, 1.25e-4, 161, 0.125, 14400)
200 lhs3, rhs3 = check_conserve(C, 3.125e-5, 641, 0.125, 14400) # 主离散
201 print('验证3 离散守恒: 0.125mm 离散  $\sum \Delta C = .6e$  vs  $-R h_m (C_R - C_{air}) dt = .6e$ , 差  $.2e$ '
202       % (lhs2, rhs2, lhs2 - rhs2))
203 print('          0.03125mm 离散  $\sum \Delta C = .6e$  vs  $-R h_m (C_R - C_{air}) dt = .6e$ , 差  $.2e$ '
204       % (lhs3, rhs3, lhs3 - rhs3))
205
206 # ----- 验证4: 渐近行为 (常数边界, 长时程, 1 mm 离散) -----
207 Ta, Ca = 41.513, 0.04986
208 ri1mm = [0, 5, 10, 15, 20] # 1 mm 离散上的 0 / 0.5 / 1 / 1.5 / 2 cm
209 rec4 = {20000: ri1mm, 50000: ri1mm, 100000: ri1mm}
210 dr = 1e-3; N = 21
211 r = np.arange(N) * dr
212 T4, C4, _, _ = solve(100000, 1.0, lambda t: Ta, lambda t: Ca, record=rec4)
213 dr = global_dr; N = global_N
214 r = np.arange(N) * dr
215 for k, tt in enumerate(rec4):
216     print('验证4 渐近: t=%ds 中心 T=%.4f (目标 %.4f), 表面 C=%.5f (目标 %.5f)'
217           % (tt, T4[k][0], Ta, C4[k][4], Ca))
218
219 # ----- 附件2 收缩数据核对 (支撑问题1忽略收缩的假设) -----
220 d2 = np.array([row for row in
221                openpyxl.load_workbook(ANNEX2, data_only=True)['Sheet1'].iter_rows(values_only=True)][1:],
222                float)
223 print('附件2: t=0 半径=%.4f cm, t=1800s 半径=%.4f cm, 末点 t=%ds 半径=%.4f cm'
224       % (d2[0, 1], d2[1, 1], d2[-1, 0], d2[-1, 1]))

```

q2_solver.py

```

1 # -*- coding: utf-8 -*-
2 """2026 CUMCM A 题 问题2 求解器
3 整个烘干过程模型 (预热平衡 + 恒温干燥两阶段烘房条件), 经验公式统一采用附录3。
4 物性随 C、T 变, 热质双向耦合, 交错 Picard + Crank--Nicolson + Thomas。
5 问题2: 0--10800 s (主离散  $\Delta r=0.125$  mm,  $\Delta t=0.25$  s) -> 表3/表4 + result2.xlsx (每 1 s、每 0.1 cm)。

```

```

6 烘房条件: 0--14400 s 用附件1 插值, 之后恒温段取附件1 末值 (50.165 °C, 0.04986)。
7 迭代: T、C 无穷范数相对变化 < 1e-8 停止 (上限 20 轮)。
8 依据: 时间步误差量级实测为 0(1e-5) (Δt 减半差值), 迭代残差须再低 2--3 个量级,
9 取 1e-8 (相对) 使迭代误差不成为精度瓶颈; 系数在迭代中取最新可用层。
10 验证: V1 时间收敛、V2 空间收敛 (0.5/0.25/0.125 mm 同 Δt)、V3 渐近。
11 """
12 import sys, os
13 import numpy as np
14 import openpyxl
15
16 try:
17     sys.stdout.reconfigure(encoding='utf-8')
18 except Exception:
19     pass
20
21 ROOT = os.path.dirname(os.path.abspath(__file__))
22
23 def _annex(name):
24     """定位赛题附件数据: 优先脚本同目录(支撑材料布局), 其次仓库目录结构。"""
25     for p in (os.path.join(ROOT, name),
26              os.path.join(ROOT, 'OriginalMaterial', 'A题', '附件', name)):
27         if os.path.exists(p):
28             return p
29     raise FileNotFoundError('未找到 %s: 请将赛题附件文件放在脚本同目录下' % name)
30
31 ANNEX1 = _annex('附件1.xlsx')
32
33 R, dr, N = 0.02, 1.25e-4, 161          # 半径 2 cm, 主离散 0.125 mm, 161 节点
34 r = np.arange(N) * dr
35 T0, C0 = 28.0, 2.55
36 h, hm = 25.0, 8e-7
37 TEND, CEND = 50.165, 0.04986          # 附件1 末值 (恒温干燥段)
38 PICARD_TOL, PICARD_MAXIT = 1e-8, 20
39
40 # 附录3 经验式 (问题2、问题3 统一采用)
41 rho23 = lambda C: 650.0 + 128.0 * C
42 cp23 = lambda C: 1450.0 + 2736.0 * C / (C + 1.0)
43 k23 = lambda C: 0.21 + 0.38 * C / (C + 1.0)
44 D23 = lambda C, T: 2.4e-3 * np.exp(-0.45 / C) * np.exp(-3850.0 / T)  # T 用开尔文
45
46 d = np.array([row for row in
47               openpyxl.load_workbook(ANNEX1, data_only=True)['Sheet1'].iter_rows(values_only=True)][1:],
48              float)
49 T_air = lambda t: np.interp(t, d[:, 0], d[:, 1]) if t <= 14400 else TEND
50 C_air = lambda t: np.interp(t, d[:, 0], d[:, 2]) if t <= 14400 else CEND
51
52 def thomas(a, b, c, rhs):
53     n = len(rhs); cp = np.zeros(n); dp = np.zeros(n)
54     cp[0] = c[0] / b[0]; dp[0] = rhs[0] / b[0]
55     for i in range(1, n):
56         m = b[i] - a[i] * cp[i - 1]; cp[i] = c[i] / m; dp[i] = (rhs[i] - a[i] * dp[i - 1]) / m
57     x = np.zeros(n); x[-1] = dp[-1]
58     for i in range(n - 2, -1, -1): x[i] = dp[i] - cp[i] * x[i + 1]
59     return x
60
61 def crank_step(u, K, Cp, beta, bc_prev, bc_next, dt):
62     """Cp_i (u^{n+1}_i - u^n_i)/dt = Lu + s; K: 每节点扩散系数; Cp: 每节点容量系数。
63     热方程: K=k(C), Cp=rho(C)c_p(C), beta=h/k_s; 质方程: K=D, Cp=1, beta=h_m/D_s。"""
64     Km = (K[:-1] + K[1:]) / 2
65     lo = np.zeros(N); di = np.zeros(N); up = np.zeros(N); s = np.zeros(N)
66     di[0] = -4 * Km[0] / dr ** 2; up[0] = 4 * Km[0] / dr ** 2
67     for i in range(1, N - 1):

```

```

68     lo[i] = Km[i - 1] * (i - 0.5) / (i * dr ** 2)
69     di[i] = -(Km[i] * (i + 0.5) + Km[i - 1] * (i - 0.5)) / (i * dr ** 2)
70     up[i] = Km[i] * (i + 0.5) / (i * dr ** 2)
71     # 表面控制体  $[r_{N-3/2}, R]$ : 表面节点  $r_{N-1}=(N-1)dr=R$  位于真实表面,
72     # Robin 条件直接取节点值: 表面通量  $-R \cdot \beta \cdot K_s \cdot (u_{N-1}-bc)$ 
73     # (热:  $-R \cdot h(T_R-T_{air})$ ; 质:  $-R \cdot h_m(C_R-C_{air})$ ), 与内部界面通量逐时间层严格守恒。
74     vhat = ((N - 1) ** 2 - (N - 1.5) ** 2) * dr ** 2 / 2
75     Bc = (N - 1) * dr * beta * K[-1]
76     lo[N - 1] = Km[N - 2] * (N - 1.5) / vhat
77     di[N - 1] = -(lo[N - 1] + Bc / vhat)
78     up[N - 1] = 0.0
79     def src(bc):
80         s = np.zeros(N); s[N - 1] = Bc * bc / vhat; return s
81     # Crank--Nicolson:  $Cp_i (u^{n+1}-u^n)/dt = 1/2(Lu^{n+1}+Lu^n) + 1/2(s^{n+1}+s^n)$ 
82     lhs_lo = -0.5 * lo
83     lhs_di = Cp / dt - 0.5 * di
84     lhs_up = -0.5 * up
85     rhs = Cp / dt * u + 0.5 * (lo * np.roll(u, 1) + di * u + up * np.roll(u, -1)) \
86         + 0.5 * (src(bc_next) + src(bc_prev))
87     rhs[0] = Cp[0] / dt * u[0] + 0.5 * (di[0] * u[0] + up[0] * u[1]) \
88         + 0.5 * (src(bc_next)[0] + src(bc_prev)[0])
89     rhs[N - 1] = Cp[N - 1] / dt * u[N - 1] \
90         + 0.5 * (lo[N - 1] * u[N - 2] + di[N - 1] * u[N - 1]) \
91         + 0.5 * (src(bc_next)[N - 1] + src(bc_prev)[N - 1])
92     return thomas(lhs_lo, lhs_di, lhs_up, rhs)
93
94 def step(T, C, t_prev, t_cur, dt):
95     """一个时间步: 交错 Picard, 每轮从  $u^n$  出发; T、C 相对变化 (无穷范数, 分母取
96     max(||u||∞, 1)) 小于  $1e-8$  或满 20 轮停止。返回 (T, C, 轮数, 退出残差)。"""
97     Tn, Cn = T.copy(), C.copy()
98     for it in range(1, PICARD_MAXIT + 1):
99         Kk = k23(C); Rc = rho23(C) * cp23(C)
100         T = crank_step(Tn, Kk, Rc, h / Kk[-1], T_air(t_prev), T_air(t_cur), dt)
101         Tk = T + 273.15
102         Dc = D23(C, Tk)
103         C = crank_step(Cn, Dc, np.ones(N), hm / Dc[-1], C_air(t_prev), C_air(t_cur), dt)
104         if it > 1:
105             rel = max(np.max(np.abs(T - To)) / max(np.max(np.abs(T)), 1.0),
106                     np.max(np.abs(C - Co)) / max(np.max(np.abs(C)), 1.0))
107             if rel < PICARD_TOL:
108                 return T, C, it, rel
109         To, Co = T.copy(), C.copy()
110     return T, C, PICARD_MAXIT, rel
111
112 def run3h(drg, Ng, dtt):
113     """前 3 h 求解, 返回 (表3, 表4, 迭代统计)。"""
114     global N, dr, r
115     N, dr, r = Ng, drg, np.arange(Ng) * drg
116     T, C = np.full(N, T0), np.full(N, C0)
117     t2 = np.arange(1800, 10801, 1800)
118     ri = [0, Ng // 4, Ng // 2, 3 * Ng // 4, Ng - 1]
119     tab3, tab4, iters = [], [], []
120     for n in range(1, int(10800 / dtt) + 1):
121         T, C, it, rel = step(T, C, (n - 1) * dtt, n * dtt, dtt)
122         iters.append(it)
123         if n * dtt in t2:
124             tab3.append(T[ri].copy()); tab4.append(C[ri].copy())
125     return np.array(tab3), np.array(tab4), np.array(iters)
126
127 # ===== 问题2: 0--10800 s, 主离散  $\Delta r=0.125$  mm,  $\Delta t=0.25$  s =====
128 tab3, tab4, iters = run3h(1.25e-4, 161, 0.25)
129 tab3r = np.round(tab3, 4); tab4r = np.round(tab4, 4)

```

```

130 print('表3 温度 (°C), 行=0.5..3.0 h, 列=0/0.5/1/1.5/2 cm:')
131 print(tab3r)
132 print('表4 水分浓度 (kg/kg):')
133 print(tab4r)
134 print('Picard 统计: 平均 %.2f 轮, 最大 %d 轮, 步数 %d' % (iters.mean(), iters.max(), len(iters)))
135
136 # result2.xlsx: 每 1 s、每 0.1 cm (0.125 mm 离散上每 8 节点一列)
137 wb = openpyxl.Workbook()
138 wsT = wb.active; wsT.title = '温度'
139 wsC = wb.create_sheet('水分浓度')
140 xcols = list(range(0, N, 8))
141 for ws in (wsT, wsC):
142     ws.cell(1, 1, '时间\\到药材中心的距离')
143     for j, i in enumerate(xcols):
144         ws.cell(1, 2 + j, round(r[i] * 100, 1))
145 T, C = np.full(N, T0), np.full(N, C0)
146 for n in range(1, 43201):
147     T, C, it, rel = step(T, C, (n - 1) * 0.25, n * 0.25, 0.25)
148     if n % 4 == 0:
149         m = n // 4
150         wsT.cell(1 + m, 1, m); wsC.cell(1 + m, 1, m)
151         for j, i in enumerate(xcols):
152             wsT.cell(1 + m, 2 + j, round(float(T[i]), 4))
153             wsC.cell(1 + m, 2 + j, round(float(C[i]), 4))
154 wb.save(os.path.join(ROOT, 'result2.xlsx'))
155 print('已写出 result2.xlsx')
156
157 # ===== 验证 =====
158 # V1 时间收敛:  $\Delta t=0.5$  vs  $0.25$  (主离散, 表3/表4 最大差)
159 tab3b, tab4b, _ = run3h(1.25e-4, 161, 0.5)
160 print('验证V1 时间收敛: 表3 最大差 %.2e °C, 表4 最大差 %.2e kg/kg'
161       % (np.max(np.abs(tab3b - tab3)), np.max(np.abs(tab4b - tab4))))
162
163 # V2 空间收敛:  $0.5$  mm 与  $0.25$  mm 两套离散 (同  $\Delta t=0.25$  s、同位置采样)
164 tab3_05, tab4_05, _ = run3h(5e-4, 41, 0.25)
165 tab3_025, tab4_025, _ = run3h(2.5e-4, 81, 0.25)
166 print('验证V2 空间收敛: 表3 最大差  $0.5 \rightarrow 0.25$ mm: %.2e °C,  $0.25 \rightarrow 0.125$ mm: %.2e °C'
167       % (np.max(np.abs(tab3_05 - tab3_025)), np.max(np.abs(tab3_025 - tab3))))
168 print('      表4 最大差  $0.5 \rightarrow 0.25$ mm: %.2e,  $0.25 \rightarrow 0.125$ mm: %.2e kg/kg'
169       % (np.max(np.abs(tab4_05 - tab4_025)), np.max(np.abs(tab4_025 - tab4))))
170
171 # V3 渐近: 常数边界 (50.165, 0.04986), 长时程
172 N, dr, r = 21, 1e-3, np.arange(21) * 1e-3
173 T, C = np.full(N, T0), np.full(N, C0)
174 for n in range(1, int(3e5 / 2.5) + 1):
175     T, C, it, rel = step(T, C, (n - 1) * 2.5, n * 2.5, 2.5)
176     if n * 2.5 in (100000, 200000, 300000):
177         print('验证V3 渐近: t=%ds 中心 T=%.4f (目标 %.4f), 表面 C=%.5f (目标 %.5f)'
178               % (n * 2.5, T[0], TEND, C[-1], CEND))

```

q3_solver.py

```

1 # -*- coding: utf-8 -*-
2 """2026 CUMCM A 题 问题3 求解器
3 整个烘干过程模型, 经验公式统一采用附录3 (物性随 C、T 变),
4 热质双向耦合, 交错 Picard + Crank--Nicolson + Thomas。
5 问题3:  $\Delta r=0.0625$  mm ( $N=321$ ),  $\Delta t=2.5$  s, 从初态直至  $\max C \leq 0.15 \rightarrow$  烘干时长 t*、表5
6       + result3.xlsx (每 60 s、每 0.1 cm)。
7 步长依据: 0.125 mm 生产离散的表 5 对齐行 Richardson 估计 (按 t* 链实测阶 p=1.22) 约  $7.8e-5$ ,

```

```

8   高于四位小数对应的  $0.5e-4$  阈值;  $0.0625\text{ mm}$  的估计 (按  $p=1.33$ ) 约  $2.7e-5$ , 达标, 故生产离散取  $0.0625\text{ mm}$ 。
9   收敛阶低于整阶二阶的原因: 表 5 最大差位于 42 h 行、1.5 cm 列 (时间-浓度剖面最陡处), 且末行
10  由  $t^*$  时间插值取值, 行收敛阶 1.1--1.3 (V3 逐格定位与  $t^*$  链给出), 按实测阶做 Richardson 估计。
11  烘房条件:  $0\text{--}14400\text{ s}$  用附件1 插值, 之后恒温段取附件1 末值 ( $50.165\text{ }^{\circ}\text{C}$ ,  $0.04986$ )。
12  迭代:  $T$ 、 $C$  无穷范数相对变化  $< 1e-8$  停止 (上限 20 轮)。
13   依据: 时间步误差量级实测为  $0(1e-5)$  ( $\Delta t$  减半差值), 迭代残差须再低 2--3 个量级,
14   取  $1e-8$  (相对) 使迭代误差不成为精度瓶颈; 系数在迭代中取最新可用层。
15  事件时间:  $g(t)=\max C(r,t)-0.15$  在相邻时间层间线性插值求根, 末行取  $t^*$  处的插值状态。
16   依据: 判据由中心点控制, 中心  $C(t)$  平滑, 线性求根误差  $O(\Delta t^2)$  远小于空间离散不确定度。
17  验证: V1 离散质量守恒 (机器精度, 末行插值后为  $1e-11$  量级)、V2 时间收敛、
18   V3 空间收敛 ( $t^*$  五点链 + 表5 逐格差定位)、V4 渐近。
19  """
20  import sys, os
21  import numpy as np
22  import openpyxl
23  from scipy.linalg import solve_banded
24
25  try:
26      sys.stdout.reconfigure(encoding='utf-8')
27  except Exception:
28      pass
29
30  ROOT = os.path.dirname(os.path.abspath(__file__))
31
32  def _annex(name):
33      """定位赛题附件数据: 优先脚本同目录(支撑材料布局), 其次仓库目录结构。"""
34      for p in (os.path.join(ROOT, name),
35               os.path.join(ROOT, 'OriginalMaterial', 'A题', '附件', name)):
36          if os.path.exists(p):
37              return p
38      raise FileNotFoundError('未找到 %s: 请将赛题附件文件放在脚本同目录下' % name)
39
40  ANNEX1 = _annex('附件1.xlsx')
41
42  R, dr, N = 0.02,  $6.25e-5$ , 321          # 半径 2 cm, 主离散  $0.0625\text{ mm}$ , 321 节点
43  r = np.arange(N) * dr
44  T0, C0 = 28.0, 2.55
45  h, hm = 25.0,  $8e-7$ 
46  TEND, CEND = 50.165, 0.04986          # 附件1 末值 (恒温干燥段)
47  CTARGET = 0.15                        # 问题3 烘干判据
48  PICARD_TOL, PICARD_MAXIT =  $1e-8$ , 20
49
50  # 附录3 经验式 (问题2、问题3 统一采用)
51  rho23 = lambda C:  $650.0 + 128.0 * C$ 
52  cp23 = lambda C:  $1450.0 + 2736.0 * C / (C + 1.0)$ 
53  k23 = lambda C:  $0.21 + 0.38 * C / (C + 1.0)$ 
54  D23 = lambda C, T:  $2.4e-3 * \exp(-0.45 / C) * \exp(-3850.0 / T)$   #  $T$  用开尔文
55
56  d = np.array([row for row in
57                openpyxl.load_workbook(ANNEX1, data_only=True)['Sheet1'].iter_rows(values_only=True)][1:],
58               float)
59  T_air = lambda t: np.interp(t, d[:, 0], d[:, 1]) if t <= 14400 else TEND
60  C_air = lambda t: np.interp(t, d[:, 0], d[:, 2]) if t <= 14400 else CEND
61
62  def thomas(a, b, c, rhs):
63      """三对角求解: 与追赶法数学等价, 调用 LAPACK 带状求解(提速, 结果不变)。"""
64      n = len(rhs)
65      ab = np.zeros((3, n))
66      ab[0, 1:] = c[:-1]
67      ab[1, :] = b
68      ab[2, :-1] = a[1:]
69      return solve_banded((1, 1), ab, rhs)

```

```

70
71 def crank_step(u, K, Cp, beta, bc_prev, bc_next, dt):
72     """Cp_i ( $u^{n+1}_i - u^n_i$ )/dt = Lu + s; K: 每节点扩散系数; Cp: 每节点容量系数。
73     热方程:  $K=k(C)$ ,  $Cp=\rho(C)c_p(C)$ ,  $\beta=h/k_s$ ; 质方程:  $K=D$ ,  $Cp=1$ ,  $\beta=h_m/D_s$ ."""
74     Km = (K[:-1] + K[1:]) / 2
75     lo = np.zeros(N); di = np.zeros(N); up = np.zeros(N); s = np.zeros(N)
76     di[0] = -4 * Km[0] / dr ** 2; up[0] = 4 * Km[0] / dr ** 2
77     for i in range(1, N - 1):
78         lo[i] = Km[i - 1] * (i - 0.5) / (i * dr ** 2)
79         di[i] = -(Km[i] * (i + 0.5) + Km[i - 1] * (i - 0.5)) / (i * dr ** 2)
80         up[i] = Km[i] * (i + 0.5) / (i * dr ** 2)
81     # 表面控制体 [ $r_{N-3/2}$ , R]: 表面节点  $r_{N-1}=(N-1)dr=R$  位于真实表面,
82     # Robin 条件直接取节点值: 表面通量  $-R \cdot \beta \cdot K_s \cdot (u_{N-1} - bc)$ 
83     # (热:  $-R \cdot h(T_R - T_{air})$ ; 质:  $-R \cdot h_m(C_R - C_{air})$ ), 与内部界面通量逐时间层严格守恒。
84     vhat = ((N - 1) ** 2 - (N - 1.5) ** 2) * dr ** 2 / 2
85     Bc = (N - 1) * dr * beta * K[-1]
86     lo[N - 1] = Km[N - 2] * (N - 1.5) / vhat
87     di[N - 1] = -(lo[N - 1] + Bc / vhat)
88     up[N - 1] = 0.0
89     def src(bc):
90         s = np.zeros(N); s[N - 1] = Bc * bc / vhat; return s
91     # Crank--Nicolson:  $Cp_i (u^{n+1}_i - u^n_i)/dt = 1/2(Lu^{n+1}_i + Lu^n_i) + 1/2(s^{n+1}_i + s^n_i)$ 
92     lhs_lo = -0.5 * lo
93     lhs_di = Cp / dt - 0.5 * di
94     lhs_up = -0.5 * up
95     rhs = Cp / dt * u + 0.5 * (lo * np.roll(u, 1) + di * u + up * np.roll(u, -1)) \
96         + 0.5 * (src(bc_next) + src(bc_prev))
97     rhs[0] = Cp[0] / dt * u[0] + 0.5 * (di[0] * u[0] + up[0] * u[1]) \
98         + 0.5 * (src(bc_next)[0] + src(bc_prev)[0])
99     rhs[N - 1] = Cp[N - 1] / dt * u[N - 1] \
100         + 0.5 * (lo[N - 1] * u[N - 2] + di[N - 1] * u[N - 1]) \
101         + 0.5 * (src(bc_next)[N - 1] + src(bc_prev)[N - 1])
102     return thomas(lhs_lo, lhs_di, lhs_up, rhs)
103
104 def step(T, C, t_prev, t_cur, dt):
105     """一个时间步: 交错 Picard, 每轮从  $u^n$  出发; T、C 相对变化 (无穷范数, 分母取
106      $\max(\|u\|_0, 1)$ ) 小于  $1e-8$  或满 20 轮停止。返回 (T, C, 轮数, 退出残差)。"""
107     Tn, Cn = T.copy(), C.copy()
108     rel = 0.0
109     for it in range(1, PICARD_MAXIT + 1):
110         Kk = k23(C); Rc = rho23(C) * cp23(C)
111         T = crank_step(Tn, Kk, Rc, h / Kk[-1], T_air(t_prev), T_air(t_cur), dt)
112         Tk = T + 273.15
113         Dc = D23(C, Tk)
114         C = crank_step(Cn, Dc, np.ones(N), hm / Dc[-1], C_air(t_prev), C_air(t_cur), dt)
115         if it > 1:
116             rel = max(np.max(np.abs(T - To)) / max(np.max(np.abs(T)), 1.0),
117                     np.max(np.abs(C - Co)) / max(np.max(np.abs(C)), 1.0))
118             if rel < PICARD_TOL:
119                 return T, C, it, rel
120             To, Co = T.copy(), C.copy()
121     return T, C, PICARD_MAXIT, rel
122
123 def run_drying(drg, Ng, dtt, sample6h=True, rows_every=None, ri10=None):
124     """全程求解直至  $\max C \leq 0.15$ , t* 在相邻时间层间插值求根。
125     返回 (tstar, tab5, Cstar, CR_hist, iters, rels, rows60)。
126     tab5: 每 6 h 一行 + 烘干结束时刻末行 (0/0.5/1/1.5/2 cm, 末行为 t* 处插值状态)。
127     rows60: rows_every 秒记一行 C[ri10] (含 t* 处插值末行), rows_every=None 时不记。"""
128     global N, dr, r
129     N, dr, r = Ng, drg, np.arange(Ng) * drg
130     T, C = np.full(N, T0), np.full(N, C0)
131     tstar, nstar = None, None

```

```

132 ri5 = [0, Ng // 4, Ng // 2, 3 * Ng // 4, Ng - 1]
133 tab5 = []
134 rows60 = []
135 CR_hist = [hm * (C0 - C_air(0.0))]
136 iters, rels = [], []
137 nmax = int(300 * 3600 / dtt)      # 最多 300 h
138 for n in range(1, nmax + 1):
139     Cprev = C.copy()
140     T, C, it, rel = step(T, C, (n - 1) * dtt, n * dtt, dtt)
141     iters.append(it); rels.append(rel)
142     CR_hist.append(hm * (C[-1] - C_air(n * dtt)))
143     if sample6h and n % int(6 * 3600 / dtt) == 0:
144         tab5.append(C[ri5].copy())
145     if rows_every is not None and n % int(rows_every / dtt) == 0:
146         rows60.append((n * dtt, C[ri10].copy()))
147     if C.max() <= CTARGET:
148         # g = max C - 0.15 在 [t_{n-1}, t_n] 间线性求根
149         gp = Cprev.max() - CTARGET
150         gn = C.max() - CTARGET
151         frac = gp / (gp - gn)
152         tstar = (n - 1) * dtt + dtt * frac
153         Cstar = Cprev + frac * (C - Cprev)
154         nstar = n
155         break
156 if tstar is None:
157     raise RuntimeError('300 h 内未达烘干判据')
158 if sample6h:
159     tab5.append(Cstar[ri5].copy())      # 末行: 烘干结束时刻 t*
160 if rows_every is not None:
161     rows60.append((tstar, Cstar[ri10].copy()))
162 return tstar, np.array(tab5), Cstar, np.array(CR_hist), np.array(iters), np.array(rels), rows60
163
164 if __name__ == "__main__":
165     # ===== 问题3: Δr=0.0625 mm, Δt=2.5 s, 直至 max C <= 0.15 =====
166     dt3 = 2.5
167     ri10 = np.arange(0, N, int(round(0.001 / dr)))      # 0:0.1:2 cm, 0.0625 mm 下每 16 节点
168     tstar, tab5, C3f, CR_hist, iters, rels, rows60 = run_drying(dr, N, dt3, rows_every=60, ri10=ri10)
169     print('问题3 烘干时长: t* = %.3f s = %.4f h (max C = %.6f)'
170           % (tstar, tstar / 3600, C3f.max()))
171     print('Picard 统计: 平均 %.2f 轮, 最大 %d 轮, 退出残差 中位 %.2e / 最大 %.2e'
172           % (iters.mean(), iters.max(), np.median(rels), rels.max()))
173     tab5r = np.round(tab5, 4)
174     print('表5 水分浓度 (kg/kg), 行=6,12,...h 直至 t*, 列=0/0.5/1/1.5/2 cm:')
175     print(tab5r)
176
177     # result3.xlsx: 每 60 s、每 0.1 cm (与上一次求解), 末行补 t* 处插值状态
178     wb = openpyxl.Workbook()
179     ws = wb.active; ws.title = 'Sheet1'
180     ws.cell(1, 1, '时间\\到药材中心的距离')
181     for j, i in enumerate(ri10):
182         ws.cell(1, 2 + j, round(r[i] * 100, 1))
183     for k, (tt, row) in enumerate(rows60):
184         ws.cell(2 + k, 1, round(tt, 1))
185         for j in range(len(ri10)):
186             ws.cell(2 + k, 2 + j, round(float(row[j]), 4))
187     wb.save(os.path.join(ROOT, 'result3.xlsx'))
188     print('已写出 result3.xlsx (%d 行)' % len(rows60))
189
190     # ===== 验证 =====
191     # V1 离散质量守恒 (全程: EvΔC vs 表面通量积分; 末行为 t* 处插值状态,
192     # 插值使闭合差由机器精度升至 1e-11 量级, 相对总失水量仍可忽略)
193     v = np.zeros(N); v[0] = dr ** 2 / 8

```

```

194 v[1:N - 1] = np.arange(1, N - 1) * dr ** 2
195 v[N - 1] = ((N - 1) ** 2 - (N - 1.5) ** 2) * dr ** 2 / 2
196 lhs = (C3f * v).sum() - C0 * v.sum()
197 rhs = -R * np.trapezoid(CR_hist, np.arange(0, len(CR_hist)) * dt3)
198 print('验证V1 离散守恒(0.0625mm):  $\Sigma \Delta C = %.6e$ ,  $-R h_m(C_R - C_{air})dt = %.6e$ , 差  $%.2e$ '
199       % (lhs, rhs, lhs - rhs))
200
201 # V2 时间收敛:  $\Delta t=2.5$  vs  $1.25$  (前 6 h 表5 首行, 主离散)
202 def row6h(dt):
203     global N, dr, r
204     N, dr, r = 321, 6.25e-5, np.arange(321) * 6.25e-5
205     T, C = np.full(N, T0), np.full(N, C0)
206     for n in range(1, int(21600 / dt) + 1):
207         T, C, it, rel = step(T, C, (n - 1) * dt, n * dt, dt)
208     ri5 = [0, 80, 160, 240, 320]
209     return C[ri5].copy()
210 r25 = row6h(2.5); r125 = row6h(1.25)
211 print('验证V2 时间收敛(6h):  $dt=2.5$  vs  $1.25$  最大差  $%.2e$  kg/kg' % np.max(np.abs(r25 - r125)))
212
213 # V3 空间收敛:  $t^*$  五点链 (0.5/0.25/0.125/0.0625/0.03125 mm, 同  $\Delta t=2.5$  s)
214 # + 表5 对齐行逐格差 (定位最大差所在行/列)
215 ts = []; tabs = []
216 for drg, Ng in ((5e-4, 41), (2.5e-4, 81), (1.25e-4, 161), (6.25e-5, 321), (3.125e-5, 641)):
217     t_, tab_, _, _, it_, rl_, _ = run_drying(drg, Ng, dt3)
218     ts.append(t_); tabs.append(tab_)
219     print('V3 链  $\Delta r=%.5f$  mm:  $t^* = %.4f$  h (%d s), 平均  $%.2f$  轮'
220           % (drg * 1e3, t_ / 3600, round(t_), it_.mean()))
221 th = [t / 3600.0 for t in ts]
222 dd1, dd2, dd3, dd4 = th[1] - th[0], th[2] - th[1], th[3] - th[2], th[4] - th[3]
223 p12, p23, p34 = np.log2(dd1 / dd2), np.log2(dd2 / dd3), np.log2(dd3 / dd4)
224 ext = th[4] + dd4 / (2 * p34 - 1.0)
225 print('V3 实测收敛阶: p12=%.3f (0.5->0.25->0.125), p23=%.3f (0.25->0.125->0.0625), '
226       ' p34=%.3f (0.125->0.0625->0.03125)' % (p12, p23, p34))
227 print('V3 按实测阶外推:  $t^* = %.4f$  h; 最细两档差 =  $%.4f$  h; 外推-最细 =  $%.4f$  h'
228       % (ext, dd4, ext - th[4]))
229 for lab, a, b in (('0.125 vs 0.0625 mm', tabs[2][: -1], tabs[3][: -1]),
230                 ('0.0625 vs 0.03125 mm', tabs[3][: -1], tabs[4][: -1])):
231     diff = np.abs(a - b)
232     i, j = np.unravel_index(np.argmax(diff), diff.shape)
233     print('V3 表5 逐格差 %s: 最大  $%.2e$  位于 第 %d 行 (%.0f h) 第 %d 列 (%.1f cm), '
234           ' 整表最大差  $%.2e$ '
235           % (lab, diff.max(), i + 1, (i + 1) * 6.0, j, j * 0.5, diff.max()))
236 print('V3 表5 末行差(0.0625 vs 0.03125):  $%.2e$  kg/kg ( $t^*$  差  $%.4f$  h) '
237       % (np.max(np.abs(tabs[3][-1] - tabs[4][-1])), th[4] - th[3]))
238
239 # V4 渐近: 常数边界 (50.165, 0.04986), 长时程
240 N, dr, r = 21, 1e-3, np.arange(21) * 1e-3
241 T, C = np.full(N, T0), np.full(N, C0)
242 for n in range(1, int(3e5 / dt3) + 1):
243     T, C, it, rel = step(T, C, (n - 1) * dt3, n * dt3, dt3)
244     if n * dt3 in (100000, 200000, 300000):
245         print('验证V4 渐近:  $t=ds$  中心  $T=%.4f$  (目标  $%.4f$ ), 表面  $C=%.5f$  (目标  $%.5f$ )'
246               % (n * dt3, T[0], TEND, C[-1], CEND))

```

q4_solver.py

```

1 # -*- coding: utf-8 -*-
2 """2026 CUMCM A 题 问题4 求解器 (含收缩)
3 附录4 经验式 (物性随 C、T 变), 热质双向耦合, 交错 Picard + Crank--Nicolson + Thomas.

```

```

4 收缩: 附件2 半径  $R(t)$  (Pchip 插值), 拉格朗日坐标  $\text{zeta} = r/R(t)$  随物料变形,
5 菲克/傅里叶定律相对固相骨架,  $\text{zeta}$  系中无对流项:
6  $\text{dC}/\text{dt} = 1/(R^2 \text{zeta}) \text{d}(\text{zeta} D \text{dC}/\text{dzeta})/\text{dzeta}$ ,
7  $\rho c_p \text{dT}/\text{dt} = 1/(R^2 \text{zeta}) \text{d}(\text{zeta} k \text{dT}/\text{dzeta})/\text{dzeta}$  ( $\text{d}/\text{dt}$  为固定  $\text{zeta}$  的随体导数)。
8 主离散:  $\Delta = 1/160$  (161 节点),  $\Delta t = 5$  s。
9 依据:  $1/40$  离散 6 h 行误差约  $1.6\text{e-}4$ ,  $1/80$  约  $5.4\text{e-}5$  (Richardson), 仍未低于四位小数
10 对应的  $0.5\text{e-}4$  阈值, 故生产离散加密至  $1/160$ 。
11 迭代: T、C 无穷范数相对变化  $< 1\text{e-}8$  停止 (上限 20 轮); 系数取迭代最新可用层, R 取半步中点。
12 事件时间:  $g(t) = \max C(r, t) - 0.15$  在相邻时间层间线性插值求根, 末行取  $t^*$  处插值状态。
13 输出: 表6 (每 6 h x  $0/0.5/1/1.5/2$  cm + 表面, 域外记 '-') + result4.xlsx (每 60 s, 22 列)。
14 固定物理距离列用 Pchip 插值 (见 interp_dist 依据)。
15 验证: V1 动边界质量守恒 (含压缩项  $2R'/R \ C \ r \ \text{dr}$ )、V2 时间收敛、V3 离散五点链 (至  $1/320$ )、V4 渐近。
16 """
17 import sys, os
18 import numpy as np
19 import openpyxl
20 from scipy.linalg import solve_banded
21 from scipy.interpolate import PchipInterpolator
22
23 try:
24     sys.stdout.reconfigure(encoding='utf-8')
25 except Exception:
26     pass
27
28 ROOT = os.path.dirname(os.path.abspath(__file__))
29
30 def _annex(name):
31     """定位赛题附件数据: 优先脚本同目录 (支撑材料布局), 其次仓库目录结构。"""
32     for p in (os.path.join(ROOT, name),
33              os.path.join(ROOT, 'OriginalMaterial', 'A题', '附件', name)):
34         if os.path.exists(p):
35             return p
36     raise FileNotFoundError('未找到 %s: 请将赛题附件文件放在脚本同目录下' % name)
37
38 ANNEX1 = _annex('附件1.xlsx')
39 ANNEX2 = _annex('附件2.xlsx')
40
41 RO, TO, CO = 0.02, 28.0, 2.55
42 h, hm = 25.0,  $8\text{e-}7$ 
43 TEND, CEND = 50.165, 0.04986 # 附件1 末值 (恒温干燥段)
44 CTARGET = 0.15 # 问题4 烘干判据
45 DZ, NZ = 1.0 / 160.0, 161 # 主离散 161 节点
46 zeta = np.arange(NZ) * DZ
47 PICARD_TOL, PICARD_MAXIT =  $1\text{e-}8$ , 20
48
49 # 附录4 经验式 ( $C < 0.02$  时取  $0.02$ , 仅为数值保险: 物理解  $C \geq C_{\text{air}} = 0.04986$ )
50 rho4 = lambda C:  $760.0 + 90.0 * C$ 
51 cp4 = lambda C:  $1850.0 + 2150.0 * C / (C + 1.0)$ 
52 k4 = lambda C:  $0.12 + 0.20 * C / (C + 1.0)$ 
53 D4 = lambda C, T:  $4.2\text{e-}4 * \text{np.exp}(-0.30 / \text{np.maximum}(C, 0.02)) * \text{np.exp}(-3850.0 / T)$  # T 开尔文
54
55 d1 = np.array([row for row in
56                openpyxl.load_workbook(ANNEX1, data_only=True)['Sheet1'].iter_rows(values_only=True)][1:],
57                float)
58 T_air = lambda t: np.interp(t, d1[:, 0], d1[:, 1]) if t <= 14400 else TEND
59 C_air = lambda t: np.interp(t, d1[:, 0], d1[:, 2]) if t <= 14400 else CEND
60
61 d2 = np.array([row for row in
62                openpyxl.load_workbook(ANNEX2, data_only=True)['Sheet1'].iter_rows(values_only=True)][1:],
63                float)
64 T2END = d2[-1, 0]; REND = d2[-1, 1] / 100.0 # 259200 s 后半径保持 1.198 cm (附件2 单位为 cm)
65 _Rp = PchipInterpolator(d2[:, 0], d2[:, 1] / 100.0, extrapolate=False)

```

```

66 RofT = lambda t: _Rp(t) if t <= T2END else REND
67 _Rpd = _Rp.derivative()
68 RpodT = lambda t: float(_Rpd(t)) if t <= T2END else 0.0
69
70 def interp_dist(C, R, dj):
71     """固定物理距离 dj 处的浓度: 在 =dj/R 处用 Pchip (三次 Hermite、保单调) 插值。
72     依据: C() 为抛物方程的光滑解 (无激波), 三次插值误差  $O(\Delta^3)$ , 远小于线性插值的
73      $O(\Delta^2)$ ; 后者在 30 h 前后的陡梯度区已主导表 6 行差 (1/80~1/160 实测  $1.58e-4$ ,
74     与线性插值误差估计同量级, 导致收敛比仅 2.5 而非 4), 故输出与收敛检验统一改用 Pchip。"""
75     return float(PchipInterpolator(zeta, C)(dj / R))
76
77 def thomas(a, b, c, rhs):
78     """三对角求解: 与追赶法数学等价, 调用 LAPACK 带状求解 (提速, 结果不变)。"""
79     n = len(rhs)
80     ab = np.zeros((3, n))
81     ab[0, 1:] = c[:-1]
82     ab[1, :] = b
83     ab[2, :-1] = a[1:]
84     return solve_banded((1, 1), ab, rhs)
85
86 def crank_step(u, K, Cp, R, beta, bc_prev, bc_next, dt):
87     """zeta 系 Crank--Nicolson 一步 (物料随体坐标, 无对流项; 扩散系数含  $1/R^2$ )。
88     beta: 表面 Robin 系数 (热  $h \cdot R/k_s$ ; 质  $h_m \cdot R/D_s$ )。"""
89     Km = (K[:-1] + K[1:]) / 2
90     R2 = R * R
91     lo = np.zeros(NZ); di = np.zeros(NZ); up = np.zeros(NZ); s = np.zeros(NZ)
92     di[0] = -4 * Km[0] / (DZ ** 2 * R2); up[0] = 4 * Km[0] / (DZ ** 2 * R2)
93     for i in range(1, NZ - 1):
94         lo[i] = Km[i - 1] * (i - 0.5) / (i * DZ ** 2 * R2)
95         di[i] = -(Km[i] * (i + 0.5) + Km[i - 1] * (i - 0.5)) / (i * DZ ** 2 * R2)
96         up[i] = Km[i] * (i + 0.5) / (i * DZ ** 2 * R2)
97     # 表面控制体 [zeta_{NZ-3/2}, 1]: 表面节点位于真实表面 zeta=1 (离散),
98     # Robin 通量直接取  $-R \cdot \beta \cdot K_s \cdot (u_{\{NZ-1\}} - bc)$ , 逐时间层严格遵守。
99     # 界面通量 =  $zeta_{\{NZ-3/2\}} \cdot D_m \cdot (u_{\{NZ-2\}} - u) / DZ = (NZ-1.5) \cdot D_m \cdot (u_{\{NZ-2\}} - u)$  (DZ 抵消)。
100     vhat = (1.0 - ((NZ - 1.5) * DZ) ** 2) / 2.0
101     Bc = beta * K[-1] # beta 已含 R: 热  $h \cdot R/k_s$ , 质  $h_m \cdot R/D_s$ 
102     lo[NZ - 1] = Km[NZ - 2] * (NZ - 1.5) / (vhat * R2)
103     di[NZ - 1] = -(lo[NZ - 1] + Bc / (vhat * R2))
104     up[NZ - 1] = 0.0
105     def src(bc):
106         s = np.zeros(NZ); s[NZ - 1] = Bc * bc / (vhat * R2); return s
107     lhs_lo = -0.5 * lo
108     lhs_di = Cp / dt - 0.5 * di
109     lhs_up = -0.5 * up
110     rhs = Cp / dt * u + 0.5 * (lo * np.roll(u, 1) + di * u + up * np.roll(u, -1)) \
111         + 0.5 * (src(bc_next) + src(bc_prev))
112     rhs[0] = Cp[0] / dt * u[0] + 0.5 * (di[0] * u[0] + up[0] * u[1]) \
113         + 0.5 * (src(bc_next)[0] + src(bc_prev)[0])
114     rhs[NZ - 1] = Cp[NZ - 1] / dt * u[NZ - 1] \
115         + 0.5 * (lo[NZ - 1] * u[NZ - 2] + di[NZ - 1] * u[NZ - 1]) \
116         + 0.5 * (src(bc_next)[NZ - 1] + src(bc_prev)[NZ - 1])
117     return thomas(lhs_lo, lhs_di, lhs_up, rhs)
118
119 def step(T, C, t_prev, t_cur, dt):
120     """一个时间步: R、R' 取半步中点, 交错 Picard, 每轮从 u^n 出发;
121     T、C 相对变化 (无穷范数, 分母取  $\max(\|u\|_\infty, 1)$ ) 小于  $1e-8$  或满 20 轮停止。"""
122     tm = 0.5 * (t_prev + t_cur)
123     R = RofT(tm)
124     Tn, Cn = T.copy(), C.copy()
125     rel = 0.0
126     for it in range(1, PICARD_MAXIT + 1):
127         Kk = k4(C); Rc = rho4(C) * cp4(C)

```

```

128     T = crank_step(Tn, Kk, Rc, R, h * R / Kk[-1], T_air(t_prev), T_air(t_cur), dt)
129     Tk = T + 273.15
130     Dc = D4(C, Tk)
131     C = crank_step(Cn, Dc, np.ones(NZ), R, hm * R / Dc[-1], C_air(t_prev), C_air(t_cur), dt)
132     if it > 1:
133         rel = max(np.max(np.abs(T - To)) / max(np.max(np.abs(T)), 1.0),
134                 np.max(np.abs(C - Co)) / max(np.max(np.abs(C)), 1.0))
135         if rel < PICARD_TOL:
136             return T, C, it, rel
137     To, Co = T.copy(), C.copy()
138     return T, C, PICARD_MAXIT, rel
139
140 def water_int(C, R):
141     """ $W = \sum v_i C_i$  (随体域离散控制体体积积分,  $v_i$  与 crank_step 一致)。"""
142     v = np.zeros(NZ); v[0] = DZ ** 2 / 8
143     v[1:NZ - 1] = zeta[1:NZ - 1] * DZ
144     v[NZ - 1] = (1.0 - ((NZ - 1.5) * DZ) ** 2) / 2.0
145     return R * R * (C * v).sum()
146
147 def run4(dzg, Nzg, dt, write_files=True):
148     """全程求解直至  $\max C \leq 0.15$  ( $t^*$  插值求根)。返回
149     (tstar, tab6, rows4, 统计)。tab6 末行为  $t^*$  处插值状态。"""
150     global NZ, DZ, zeta
151     NZ, DZ, zeta = Nzg, dzg, np.arange(Nzg) * dzg
152     T, C = np.full(NZ, T0), np.full(NZ, C0)
153     dist = np.array([0.0, 0.005, 0.01, 0.015, 0.02]) # 表6 固定距离列 (m)
154     xlsx_d = np.arange(0.0, 0.02001, 0.001) # result4 0:0.1:2 cm (m)
155     tab6, rows4 = [], []
156     Whist, Cshist, Rhist, Rphist = [], [], [], []
157     iters, rels = [], []
158     tstar, nstar = None, None
159     nmax = int(300 * 3600 / dt)
160     for n in range(1, nmax + 1):
161         tn = n * dt
162         Cprev = C.copy()
163         T, C, it, rel = step(T, C, tn - dt, tn, dt)
164         iters.append(it); rels.append(rel)
165         R = RofT(tn); Rp = RpofT(tn)
166         W = water_int(C, R)
167         Whist.append(W); Cshist.append(C[-1]); Rhist.append(R); Rphist.append(Rp)
168         if n % (21600 // dt) == 0: # 每 6 h 记表6 一行
169             row = []
170             for dj in dist:
171                 row.append(interp_dist(C, R, dj) if dj <= R else np.nan)
172                 row.append(C[-1]) # 表面列
173             tab6.append(row)
174         if write_files and n % (60 // dt) == 0: # 每 60 s 记 result4 一行
175             row = []
176             for dj in xlsx_d:
177                 row.append(interp_dist(C, R, dj) if dj <= R else None)
178             row.append(C[-1])
179             rows4.append((tn, row))
180         if C.max() <= CTARGET:
181             gp = Cprev.max() - CTARGET
182             gn = C.max() - CTARGET
183             frac = gp / (gp - gn)
184             tstar = (n - 1) * dt + dt * frac
185             Cstar = Cprev + frac * (C - Cprev)
186             nstar = n
187             break
188     if tstar is None:
189         raise RuntimeError('300 h 内未达烘干判据')

```

```

190 R = RofT(tstar)
191 row = [interp_dist(Cstar, R, dj) if dj <= R else np.nan for dj in dist] + [Cstar[-1]]
192 tab6.append(row)
193 if write_files:
194     row = [interp_dist(Cstar, R, dj) if dj <= R else None for dj in xlsx_d] + [Cstar[-1]]
195     rows4.append((tstar, row))
196 return tstar, np.array(tab6), (rows4 if write_files else None), \
197     np.array(iters), np.array(rels), (Whist, Cshist, Rhist, Rphist), nstar, Cstar
198
199 if __name__ == "__main__":
200     # ===== 问题4:  $\Delta=1/160$ ,  $dt=5$  s, 直到  $\max C \leq 0.15$  =====
201     DT = 5.0
202     tstar, tab6, rows4, iters, rels, hist, nstar, Cstar = run4(1.0 / 160.0, 161, DT)
203     R = RofT(tstar)
204     print('问题4 烘干时长:  $t^* = %.3f$  s =  $%.4f$  h (max C =  $%.6f$ , 此时 R =  $%.4f$  cm)'
205           % (tstar, tstar / 3600, Cstar.max(), R * 100))
206     print('Picard 统计: 平均  $%.2f$  轮, 最大  $%d$  轮, 退出残差 中位  $%.2e$  / 最大  $%.2e$ '
207           % (iters.mean(), iters.max(), np.median(rels), rels.max()))
208     np.set_printoptions(precision=4, suppress=True)
209     print('表6 水分浓度 (kg/kg), 行=6,12,...h 直至  $t^*$ , 列=0/0.5/1/1.5/2 cm + 表面:')
210     for k, row in enumerate(tab6):
211         rr = ' '.join(' --' if np.isnan(v) else '%6.4f' % v for v in row)
212         trow = (k + 1) * 6.0 if k + 1 < len(tab6) else tstar / 3600.0
213         print('%7.1fh |%s' % (trow, rr))
214
215     wb = openpyxl.Workbook()
216     ws = wb.active; ws.title = 'Sheet1'
217     ws.cell(1, 1, '时间\\到药材中心的距离')
218     xlsx_d = np.arange(0.0, 0.02001, 0.001)
219     for j in range(len(xlsx_d)):
220         ws.cell(1, 2 + j, round(xlsx_d[j] * 100, 1))
221     ws.cell(1, 2 + len(xlsx_d), '药材表面')
222     for k, (tt, row) in enumerate(rows4):
223         ws.cell(2 + k, 1, round(tt, 1))
224         for j, v in enumerate(row):
225             if v is not None:
226                 ws.cell(2 + k, 2 + j, round(v, 4))
227     wb.save(os.path.join(ROOT, 'result4.xlsx'))
228     print('已写出 result4.xlsx (%d 行)' % len(rows4))
229
230     # ===== 验证 =====
231     Whist, Cshist, Rhist, Rphist = hist
232     # V1 动边界质量守恒:  $d/dt \ C \ r \ dr = 2R'/R \cdot C \ r \ dr - R \cdot h_m(C_s - C_{air})$ 
233     _NZ_save, _DZ_save, _zeta_save = NZ, DZ, zeta
234     NZ, DZ, zeta = 161, 1.0 / 160.0, np.arange(161) * (1.0 / 160.0)
235     W0 = water_int(np.full(NZ, CO), R0)
236     NZ, DZ, zeta = _NZ_save, _DZ_save, _zeta_save
237     lhs = Whist[-1] - W0
238     tt = np.arange(1, nstar + 1) * DT
239     f1 = [2 * rp / r * w for rp, r, w in zip(Rphist, Rhist, Whist)]
240     f2 = [-r * hm * (cs - C_air(t)) for r, cs, t in zip(Rhist, Cshist, tt)]
241     rhs = np.trapezoid([a + b for a, b in zip(f1, f2)], tt)
242     print('验证V1 离散守恒:  $\Sigma \Delta C = %.6e$ ,  $[2R \backslash / R \cdot W - R \cdot h_m(C_R - C_{air})]dt = %.6e$ , 差  $%.2e$ '
243           % (lhs, rhs, lhs - rhs))
244
245     # V2 时间收敛: 前 6 h,  $dt=5$  vs 2.5 (同一离散 1/160)
246     def run6h(dt):
247         T, C = np.full(NZ, T0), np.full(NZ, CO)
248         for n in range(1, int(21600 / dt) + 1):
249             T, C, it, rel = step(T, C, (n - 1) * dt, n * dt, dt)
250             R = RofT(21600)
251             dist = np.array([0.0, 0.005, 0.01, 0.015, 0.02])

```

```

252     row = [interp_dist(C, R, dj) if dj <= R else np.nan for dj in dist] + [C[-1]]
253     return np.array(row)
254 r5 = run6h(5.0); r25 = run6h(2.5)
255 print('验证V2 时间收敛(6h 行): dt=5 vs 2.5 最大差 %.2e' % np.nanmax(np.abs(r5 - r25)))
256
257 # V3 空间收敛五点链: 1/20, 1/40, 1/80, 1/160, 1/320 (同 dt=5 s)
258 ts = []; t6s = []
259 for dzg, nzg in ((1.0 / 20, 21), (1.0 / 40, 41), (1.0 / 80, 81),
260                 (1.0 / 160, 161), (1.0 / 320, 321)):
261     t_, tab_, _, it_, rl_, _, _ = run4(dzg, nzg, DT, write_files=False)
262     ts.append(t_); t6s.append(tab_)
263     print('V3 链 Δ=1/%d: t* = %.4f h (%d s), 平均 %.2f 轮'
264           % (nzg - 1, t_ / 3600, round(t_), it_.mean()))
265 th = [t / 3600.0 for t in ts]
266 dd1, dd2, dd3, dd4 = th[1] - th[0], th[2] - th[1], th[3] - th[2], th[4] - th[3]
267 p12, p23, p34 = np.log2(dd1 / dd2), np.log2(dd2 / dd3), np.log2(dd3 / dd4)
268 ext = th[4] + dd4 / (2 ** p34 - 1.0)
269 print('V3 实测收敛阶: p12=%.3f (1/20->1/40->1/80), p23=%.3f (1/40->1/80->1/160), '
270       ' p34=%.3f (1/80->1/160->1/320)' % (p12, p23, p34))
271 print('V3 按实测阶外推: t* = %.4f h; 最细两档差 = %.4f h; 外推-最细 = %.4f h'
272       % (ext, dd4, ext - th[4]))
273 dab = np.nanmax(np.abs(t6s[2][:-1] - t6s[3][:-1]))
274 dbc = np.nanmax(np.abs(t6s[3][:-1] - t6s[4][:-1]))
275 print('V3 表6 对齐行最大差: 1/80->1/160: %.2e, 1/160->1/320: %.2e kg/kg' % (dab, dbc))
276 print('V3 表6 未行差(1/160 vs 1/320): %.2e kg/kg (t* 差 %.4f h)'
277       % (np.nanmax(np.abs(t6s[3][-1] - t6s[4][-1])), th[4] - th[3]))
278
279 # V4 渐近: 常数边界, 长时程 (R 仍按附件2 收缩并保持 1.198 cm)
280 NZ, DZ, zeta = 161, 1.0 / 160.0, np.arange(161) * (1.0 / 160.0)
281 T, C = np.full(NZ, T0), np.full(NZ, C0)
282 for n in range(1, int(2.5e5 / DT) + 1):
283     T, C, it, rel = step(T, C, (n - 1) * DT, n * DT, DT)
284     if n * DT in (50000, 150000, 250000):
285         print('验证V4 渐近: t=%ds 中心 T=%.4f (目标 %.4f), 表面 C=%.5f (目标 %.5f), R=%.4f cm'
286               % (n * DT, T[0], TEND, C[-1], CEND, RofT(n * DT) * 100))

```

表 7 消融 _ 表 8 灵敏度.py

```

1  # -*- coding: utf-8 -*-
2  """2026 CUMCM A 题 表8 参数灵敏度重算 (新求解器: Picard<1e-8、t* 根插值)
3
4  问题3: 径向离散 0.25 mm (N=81)、dt=2.5 s, 附录3 物性、固定 R;
5  问题4: 离散 1/80 (N=81)、dt=5 s, 附录4 物性、附件2 收缩 R(t)。
6  变体: Ta±2 °C、Ca±20%、h±50%、hm±50% (相对题给值), 其余参数保持题给值。
7
8  依据: 灵敏度取两变体 t* 之差, 对空间离散不敏感, 用验证离散即可 (与 7.7 节说明一致);
9  迭代判据改变引起的 t* 系统性偏移 (实测 <0.001 h) 在差分中相消, 故本表只需重算一遍。
10 """
11 import sys, os
12 import numpy as np
13 import openpyxl
14 from scipy.linalg import solve_banded
15 from scipy.interpolate import PchipInterpolator
16
17 try:
18     sys.stdout.reconfigure(encoding='utf-8')
19 except Exception:
20     pass
21

```

```

22 ROOT = os.path.dirname(os.path.abspath(__file__))
23
24 def _annex(name):
25     """定位赛题附件数据: 优先脚本同目录(支撑材料布局), 其次仓库目录结构。"""
26     for p in (os.path.join(ROOT, name),
27              os.path.join(ROOT, 'OriginalMaterial', 'A题', '附件', name)):
28         if os.path.exists(p):
29             return p
30     raise FileNotFoundError('未找到 %s: 请将赛题附件文件放在脚本同目录下' % name)
31
32 ANNEX1 = _annex('附件1.xlsx')
33 ANNEX2 = _annex('附件2.xlsx')
34
35 RO, TO, CO = 0.02, 28.0, 2.55
36 TEND, CEND = 50.165, 0.04986
37 CTARGET = 0.15
38 PICARD_TOL, PICARD_MAXIT = 1e-8, 20
39
40 # ---- 附件数据 ----
41 d1 = np.array([row for row in
42                openpyxl.load_workbook(ANNEX1, data_only=True)['Sheet1'].iter_rows(values_only=True)][1:],
43               float)
44 d2 = np.array([row for row in
45                openpyxl.load_workbook(ANNEX2, data_only=True)['Sheet1'].iter_rows(values_only=True)][1:],
46               float)
47 T2END = d2[-1, 0]; REnd = d2[-1, 1] / 100.0
48 _Rp = PchipInterpolator(d2[:, 0], d2[:, 1] / 100.0, extrapolate=False)
49
50 # ---- 变体参数 (模块级, 运行中修改) ----
51 P = dict(dToff=0.0, cscale=1.0, h=25.0, hm=8e-7)
52 T_air = lambda t: (np.interp(t, d1[:, 0], d1[:, 1]) + P['dToff']) if t <= 14400 else TEND + P['dToff']
53 C_air = lambda t: (np.interp(t, d1[:, 0], d1[:, 2]) * P['cscale']) if t <= 14400 else CEND * P['cscale']
54
55 # 附录3 / 附录4 物性
56 rho3 = lambda C: 650.0 + 128.0 * C
57 cp3 = lambda C: 1450.0 + 2736.0 * C / (C + 1.0)
58 k3 = lambda C: 0.21 + 0.38 * C / (C + 1.0)
59 D3 = lambda C, T: 2.4e-3 * np.exp(-0.45 / C) * np.exp(-3850.0 / T)
60 rho4 = lambda C: 760.0 + 90.0 * C
61 cp4 = lambda C: 1850.0 + 2150.0 * C / (C + 1.0)
62 k4 = lambda C: 0.12 + 0.20 * C / (C + 1.0)
63 D4 = lambda C, T: 4.2e-4 * np.exp(-0.30 / np.maximum(C, 0.02)) * np.exp(-3850.0 / T)
64
65 def thomas(a, b, c, rhs):
66     n = len(rhs)
67     ab = np.zeros((3, n))
68     ab[0, 1:] = c[:-1]
69     ab[1, :] = b
70     ab[2, :-1] = a[1:]
71     return solve_banded((1, 1), ab, rhs)
72
73 def crank_step(u, K, Cp, beta, bc_prev, bc_next, dt, NZ, DZ, R):
74     """径向 (R 固定) 或 系 (R 为当前半径) 的 Crank--Nicolson 一步, 表面节点在真实表面。"""
75     Km = (K[:-1] + K[1:]) / 2
76     R2 = R * R
77     lo = np.zeros(NZ); di = np.zeros(NZ); up = np.zeros(NZ)
78     di[0] = -4 * Km[0] / (DZ ** 2 * R2); up[0] = 4 * Km[0] / (DZ ** 2 * R2)
79     for i in range(1, NZ - 1):
80         lo[i] = Km[i - 1] * (i - 0.5) / (i * DZ ** 2 * R2)
81         di[i] = -(Km[i] * (i + 0.5) + Km[i - 1] * (i - 0.5)) / (i * DZ ** 2 * R2)
82         up[i] = Km[i] * (i + 0.5) / (i * DZ ** 2 * R2)
83     vhat = (1.0 - ((NZ - 1.5) * DZ) ** 2) / 2.0

```

```

84     Bc = beta * K[-1]
85     lo[NZ - 1] = Km[NZ - 2] * (NZ - 1.5) / (vhat * R2)
86     di[NZ - 1] = -(lo[NZ - 1] + Bc / (vhat * R2))
87     up[NZ - 1] = 0.0
88     def src(bc):
89         s = np.zeros(NZ); s[NZ - 1] = Bc * bc / (vhat * R2); return s
90     lhs_lo = -0.5 * lo
91     lhs_di = Cp / dt - 0.5 * di
92     lhs_up = -0.5 * up
93     rhs = Cp / dt * u + 0.5 * (lo * np.roll(u, 1) + di * u + up * np.roll(u, -1)) \
94         + 0.5 * (src(bc_next) + src(bc_prev))
95     rhs[0] = Cp[0] / dt * u[0] + 0.5 * (di[0] * u[0] + up[0] * u[1]) \
96         + 0.5 * (src(bc_next)[0] + src(bc_prev)[0])
97     rhs[NZ - 1] = Cp[NZ - 1] / dt * u[NZ - 1] \
98         + 0.5 * (lo[NZ - 1] * u[NZ - 2] + di[NZ - 1] * u[NZ - 1]) \
99         + 0.5 * (src(bc_next)[NZ - 1] + src(bc_prev)[NZ - 1])
100     return thomas(lhs_lo, lhs_di, lhs_up, rhs)
101
102 def step(T, C, t_prev, t_cur, dt, NZ, DZ, R, rhoF, cpF, kF, DF):
103     """交错 Picard, T, C 相对变化 < 1e-8 或满 20 轮停止。"""
104     Tn, Cn = T.copy(), C.copy()
105     rel = 0.0
106     for it in range(1, PICARD_MAXIT + 1):
107         Kk = kF(C); Rc = rhoF(C) * cpF(C)
108         T = crank_step(Tn, Kk, Rc, P['h'] * R / Kk[-1], T_air(t_prev), T_air(t_cur), dt, NZ, DZ, R)
109         Tk = T + 273.15
110         Dc = DF(C, Tk)
111         C = crank_step(Cn, Dc, np.ones(NZ), P['hm'] * R / Dc[-1], C_air(t_prev), C_air(t_cur), dt, NZ, DZ, R)
112         if it > 1:
113             rel = max(np.max(np.abs(T - To)) / max(np.max(np.abs(T)), 1.0),
114                     np.max(np.abs(C - Co)) / max(np.max(np.abs(C)), 1.0))
115             if rel < PICARD_TOL:
116                 return T, C, it, rel
117         To, Co = T.copy(), C.copy()
118     return T, C, PICARD_MAXIT, rel
119
120 def run_case(appendix, shrink, NZ, DZ, dt):
121     """返回 t*(s)。shrink=False 时 RR0; True 时 R(t) 取半步中点。"""
122     PF = (rho3, cp3, k3, D3) if appendix == 3 else (rho4, cp4, k4, D4)
123     RofT = (lambda t: float(_Rp(t)) if t <= T2END else REND) if shrink else (lambda t: R0)
124     T, C = np.full(NZ, T0), np.full(NZ, C0)
125     nmax = int(300 * 3600 / dt)
126     for n in range(1, nmax + 1):
127         tm = (n - 0.5) * dt
128         Cprev = C.copy()
129         T, C, it, rel = step(T, C, (n - 1) * dt, n * dt, dt, NZ, DZ, RofT(tm), *PF)
130         if C.max() <= CTARGET:
131             gp = Cprev.max() - CTARGET
132             gn = C.max() - CTARGET
133             return (n - 1) * dt + dt * gp / (gp - gn)
134     raise RuntimeError('300 h 内未达判据')
135
136 VARIANTS = [('基准', dict()),
137             ('Ta+2', dict(dToff=2.0)), ('Ta-2', dict(dToff=-2.0)),
138             ('Ca+20%', dict(cscale=1.2)), ('Ca-20%', dict(cscale=0.8)),
139             ('h+50%', dict(h=37.5)), ('h-50%', dict(h=12.5)),
140             ('hm+50%', dict(hm=1.2e-6)), ('hm-50%', dict(hm=4e-7))]
141
142 if __name__ == '__main__':
143     # ===== 表7: 物性×几何 2×2 消融 (同一验证离散 Δ=1/160, Δt=5 s; 与 7.5 节一致) =====
144     print('===== 表7 消融 (Δ=1/160, Δt=5 s) =====')
145     ts = {}

```

```

146     for name, ap, sh in (('附录3×固定', 3, False), ('附录3×收缩', 3, True),
147                          ('附录4×固定', 4, False), ('附录4×收缩', 4, True)):
148         ts[name] = run_case(ap, sh, 161, 1.0 / 160.0, 5.0)
149         print('%s: t* = %.4f h (%d s)' % (name, ts[name] / 3600, round(ts[name])))
150     print('收缩净效应: 附录3 Δt* = %.4f h (%.1f%%), 附录4 Δt* = %.4f h (%.1f%%)'
151           % ((ts['附录3×收缩'] - ts['附录3×固定']) / 3600,
152              (ts['附录3×收缩'] / ts['附录3×固定'] - 1) * 100,
153              (ts['附录4×收缩'] - ts['附录4×固定']) / 3600,
154              (ts['附录4×收缩'] / ts['附录4×固定'] - 1) * 100))
155     print()
156     for label, cfg in (('问题3 ( 离散 1/80 物理 0.25 mm, dt=2.5 s) ',
157                       dict(appendix=3, shrink=False, NZ=81, DZ=1.0/80.0, dt=2.5)),
158                       ('问题4 (Δ=1/80, dt=5 s) ',
159                        dict(appendix=4, shrink=True, NZ=81, DZ=1.0/80.0, dt=5.0))):
160         print('==== %s =====' % label)
161         rows = []
162         base = None
163         for name, v in VARIANTS:
164             for k in P: # 重置为基准再叠加变体
165                 P[k] = dict(dToff=0.0, cscale=1.0, h=25.0, hm=8e-7)[k]
166                 P.update(v)
167                 t = run_case(**cfg)
168                 rows.append((name, t))
169                 if name == '基准':
170                     base = t
171         for name, t in rows:
172             print('%s: t* = %.4f h, Δt* = %+.4f h (%+.1f%%)'
173                   % (name, t / 3600, (t - base) / 3600, (t / base - 1) * 100))
174         print()

```

verify_q1.py

```

1  # -*- coding: utf-8 -*-
2  """问题1 补充验证:表2 各单元格 0.125/0.0625/0.03125 mm 三套离散的差值定位,
3  判断四位小数可靠性的实际薄弱位置(评审口径:Richardson E(h/2) |u_h-u_{h/2}|/3)。
4  """
5  import sys
6  sys.stdout.reconfigure(encoding='utf-8')
7  sys.path.insert(0, __file__.rsplit('\\', 1)[0] if '\\' in __file__ else '.')
8  import numpy as np
9  import q1_solver as Q
10
11  t_ev = [100, 300, 600, 900, 1200, 1500, 1800]
12
13  old = (Q.dr, Q.N)
14  # 0.125 mm 粗离散 (Δt=0.125 s)
15  Q.dr, Q.N = 1.25e-4, 161
16  _, C125, _, _ = Q.solve(14400, 0.125, Q.T_air, Q.C_air,
17                           record={tt: [0, 40, 80, 120, 160] for tt in t_ev})
18  # 0.0625 mm 中离散 (Δt=0.0625 s, 时空同时折半)
19  Q.dr, Q.N = 6.25e-5, 321
20  _, C62, _, _ = Q.solve(28800, 0.0625, Q.T_air, Q.C_air,
21                           record={tt: [0, 80, 160, 240, 320] for tt in t_ev})
22  # 0.03125 mm 主离散 (Δt=0.0625 s, 与生产一致)
23  Q.dr, Q.N = 3.125e-5, 641
24  _, C31, _, _ = Q.solve(57600, 0.0625, Q.T_air, Q.C_air,
25                           record={tt: [0, 160, 320, 480, 640] for tt in t_ev})
26  Q.dr, Q.N = old
27

```

```

28 d1 = np.abs(C62 - C31) # 0.0625 -> 0.03125
29 d2 = np.abs(C125 - C62) # 0.125 -> 0.0625
30 cols = ['0', '0.5', '1', '1.5', '2']
31 print('表2 各单元格 |C(0.0625mm)-C(0.03125mm)| (行=时间s, 列=0/0.5/1/1.5/2cm):')
32 for tt, row in zip(t_ev, d1):
33     print('%5d | ' % tt + ' '.join('%2e' % v for v in row))
34 i, j = np.unravel_index(d1.argmax(), d1.shape)
35 print('最大差 %2e 位于 t=%ds, 列=%scm; Richardson 误差(0.03125mm) %2e'
36       % (d1.max(), t_ev[i], cols[j], d1.max() / 3))
37 print()
38 print('0.125mm->0.0625mm 最大差 %2e (Richardson 误差(0.0625mm) %2e)'
39       % (d2.max(), d2.max() / 3))
40 print()
41 print('表2 末行(1800s) 0.125/0.0625/0.03125mm:', np.round(C125[-1], 4),
42       np.round(C62[-1], 4), np.round(C31[-1], 4))
43 print('表2 首行(100s) 0.125/0.0625/0.03125mm:', np.round(C125[0], 4),
44       np.round(C62[0], 4), np.round(C31[0], 4))

```

verify_q3.py

```

1 # -*- coding: utf-8 -*-
2 """问题3 验证与增强脚本(评审报告问题2/14 的补充):
3 A) Picard 两轮耦合残差量化; B) t* 三档离散(0.5/0.25/0.125 mm)+ Richardson 外推;
4 C) 灵敏度:Ta±2°C、Ca±20%、h±50%、hm±50% 对 t*。
5 复用 q3_solver 的函数(q3_solver 主流程已加 __main__ 守卫)。
6 """
7 import sys
8 sys.stdout.reconfigure(encoding='utf-8')
9 sys.path.insert(0, __file__.rsplit('\\', 1)[0] if '\\' in __file__ else '.')
10 import numpy as np
11 import q3_solver as Q
12
13 DT3 = 2.5
14 CTARGET = 0.15
15
16
17 def full_run(drg, ng, dt=DT3, Ta=None, Ca=None, h=None, hm=None, tag=''):
18     """全程求解至 max C<=0.15,返回 t*。参数覆盖用模块全局临时替换。"""
19     old = (Q.TEND, Q.CEND, Q.h, Q.hm)
20     if Ta is not None:
21         Q.TEND = Ta
22     if Ca is not None:
23         Q.CEND = Ca
24     if h is not None:
25         Q.h = h
26     if hm is not None:
27         Q.hm = hm
28     tstar, _, _, _ = Q.run_drying(drg, ng, dt, sample6h=False)
29     Q.TEND, Q.CEND, Q.h, Q.hm = old
30     return tstar
31
32
33 if __name__ == '__main__':
34     print('==== A) Picard 两轮耦合残差 (0.25mm 离散, dt=2.5) =====')
35     Q.N, Q.dr, Q.r = 81, 2.5e-4, np.arange(81) * 2.5e-4
36     T, C = np.full(81, Q.T0), np.full(81, Q.C0)
37     for n in range(1, int(60000 / DT3) + 1):
38         tn, tp = n * DT3, (n - 1) * DT3
39         Tn, Cn = T.copy(), C.copy()

```

```

40     Kk = Q.k23(C); Rc = Q.rho23(C) * Q.cp23(C)
41     T1 = Q.crank_step(Tn, Kk, Rc, Q.h / Kk[-1], Q.T_air(tp), Q.T_air(tn), DT3)
42     Dc1 = Q.D23(C, T1 + 273.15)
43     C1 = Q.crank_step(Cn, Dc1, np.ones(81), Q.hm / Dc1[-1], Q.C_air(tp), Q.C_air(tn), DT3)
44     Kk2 = Q.k23(C1); Rc2 = Q.rho23(C1) * Q.cp23(C1)
45     T2 = Q.crank_step(Tn, Kk2, Rc2, Q.h / Kk2[-1], Q.T_air(tp), Q.T_air(tn), DT3)
46     Dc2 = Q.D23(C1, T2 + 273.15)
47     C2 = Q.crank_step(Cn, Dc2, np.ones(81), Q.hm / Dc2[-1], Q.C_air(tp), Q.C_air(tn), DT3)
48     dC_pic = np.max(np.abs(C2 - C1)); dC_step = np.max(np.abs(C2 - Cn))
49     dT_pic = np.max(np.abs(T2 - T1))
50     if tn in (3600, 21600, 129600, 190800):
51         print('t=%6ds: Picard |ΔC|=%.3e (步变化 %.3e, 比 %.1e) | ΔT|=%.3e'
52               % (tn, dC_pic, dC_step, dC_pic / dC_step, dT_pic))
53     T, C = T2, C2
54     if C.max() <= CTARGET:
55         break
56
57     print('==== B) t* 三离散收敛 + 外推 ====')
58     t05 = full_run(5e-4, 41, tag='0.5mm')
59     t025 = full_run(2.5e-4, 81, tag='0.25mm')
60     t0125 = full_run(1.25e-4, 161, tag='0.125mm')
61     print('t*(0.5mm)=%.4f h   t*(0.25mm)=%.4f h   t*(0.125mm)=%.4f h'
62           % (t05 / 3600, t025 / 3600, t0125 / 3600))
63     ext = (4 * t025 - t05) / 3
64     ext2 = (4 * t0125 - t025) / 3
65     print('Richardson(0.5,0.25): %.4f h ; (0.25,0.125): %.4f h ; |ext2-ext|=%.4f h'
66           % (ext / 3600, ext2 / 3600, abs(ext2 - ext) / 3600))
67
68     print('==== C) 灵敏度 (0.25mm, dt=2.5) ====')
69     base = t025
70     for tag, kw in [('Ta+2', dict(Ta=52.165)), ('Ta-2', dict(Ta=48.165)),
71                   ('Ca+20%', dict(Ca=0.04986 * 1.2)), ('Ca-20%', dict(Ca=0.04986 * 0.8)),
72                   ('h+50%', dict(h=37.5)), ('h-50%', dict(h=12.5)),
73                   ('hm+50%', dict(hm=1.2e-6)), ('hm-50%', dict(hm=4e-7))]:
74         ts = full_run(2.5e-4, 81, tag=tag, **kw)
75         print('%-8s t* = %.4f h (Δ %.3f h, %+.1f%%)' % (tag, ts / 3600, (ts - base) / 3600,
76               (ts - base) / base * 100))

```

verify_q4.py

```

1  # -*- coding: utf-8 -*-
2  """问题4 验证与增强脚本（评审报告问题4/5/6/14 的复验 + 消融 + 灵敏度）。
3  复用 q4_solver 的函数（q4_solver 主流程已加 __main__ 守卫,导入不触发求解）。
4  A) 6h 行离散收敛 N=21/41/81; B) N=81 时间收敛 dt=5 vs 2.5;
5  C) Picard 两轮耦合残差量化; D) N=81 生产复算(t*, 表6、V1 守恒);
6  E) t* 离散收敛 21/41/81 + Richardson 外推; F) 消融:附录4 + 固定 R=2cm;
7  G) 灵敏度:Ta±2、Ca±20%、h±50%、hm±50% 对 t*。
8  """
9  import sys
10 sys.stdout.reconfigure(encoding='utf-8')
11 sys.path.insert(0, __file__.rsplit('\\', 1)[0] if '\\' in __file__ else '.')
12 import numpy as np
13 import q4_solver as Q
14
15 DT = 5.0
16 CTARGET = 0.15
17 DIST = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
18
19

```

```

20 def row6h(dz, nz, dt=DT):
21     """前 6 h 的表6 行(含表面列),指定离散。"""
22     old = (Q.NZ, Q.DZ, Q.zeta)
23     Q.NZ, Q.DZ = nz, dz
24     Q.zeta = np.arange(nz) * dz
25     T, C = np.full(nz, Q.T0), np.full(nz, Q.CO)
26     for n in range(1, int(21600 / dt) + 1):
27         T, C = Q.step(T, C, (n - 1) * dt, n * dt, dt)
28     R = Q.RofT(21600)
29     row = [float(np.interp(d / R, Q.zeta, C)) if d <= R else np.nan for d in DIST] + [C[-1]]
30     Q.NZ, Q.DZ, Q.zeta = old
31     return np.array(row)
32
33
34 def full_run(dz, nz, dt=DT, fixed_R=False, Ta=None, Ca=None, h=None, hm=None,
35             tag='', sample=False):
36     """全程求解至 max C<=0.15。fixed_R: 消融实验(附录4 物性 + R 恒为 2cm)。
37     Ta/Ca/h/hm 传 None 用默认;非 None 时临时覆盖 q4_solver 的模块全局。"""
38     old = (Q.NZ, Q.DZ, Q.zeta, Q.TEND, Q.CEND, Q.h, Q.hm)
39     Q.NZ, Q.DZ = nz, dz
40     Q.zeta = np.arange(nz) * dz
41     if Ta is not None:
42         Q.TEND = Ta
43     if Ca is not None:
44         Q.CEND = Ca
45     if h is not None:
46         Q.h = h
47     if hm is not None:
48         Q.hm = hm
49     R0c = 0.02
50     T, C = np.full(nz, Q.T0), np.full(nz, Q.CO)
51     tstar = None
52     Whist, Cshist, Rhist, Rphist, tt_hist = [], [], [], [], []
53     tab6 = []
54     nmax = int(300 * 3600 / dt)
55     for n in range(1, nmax + 1):
56         tn = n * dt
57         tp = tn - dt
58         tm = 0.5 * (tp + tn)
59         R = R0c if fixed_R else Q.RofT(tm)
60         Tn, Cn = T.copy(), C.copy()
61         for _ in range(2):
62             Kk = Q.k4(C); Rc = Q.rho4(C) * Q.cp4(C)
63             T = Q.crank_step(Tn, Kk, Rc, R, Q.h * R / Kk[-1], Q.T_air(tp), Q.T_air(tn), dt)
64             Dc = Q.D4(C, T + 273.15)
65             C = Q.crank_step(Cn, Dc, np.ones(nz), R, Q.hm * R / Dc[-1], Q.C_air(tp), Q.C_air(tn), dt)
66             W = Q.water_int(C, R)
67             Whist.append(W); Cshist.append(C[-1]); Rhist.append(R); Rphist.append(0.0 if fixed_R else Q.RpofT(tn))
68             tt_hist.append(tn)
69             if n % int(6 * 3600 / dt) == 0:
70                 row = [float(np.interp(d / R, Q.zeta, C)) if d <= R else np.nan for d in DIST] + [C[-1]]
71                 tab6.append(row)
72                 if C.max() <= CTARGET:
73                     tstar = tn
74                     row = [float(np.interp(d / R, Q.zeta, C)) if d <= R else np.nan for d in DIST] + [C[-1]]
75                     tab6.append(row)
76                     break
77             if tstar is None:
78                 raise RuntimeError(tag + '300 h 内未达判据')
79             # V1 动边界守恒(同 q4_solver 主流程口径)
80             W0 = Q.water_int(np.full(nz, Q.CO), R0c)
81             lhs = Whist[-1] - W0

```

```

82     tt = np.array(tt_hist)
83     f1 = [2 * rp / r * w for rp, r, w in zip(Rphist, Rhist, Whist)]
84     f2 = [-r * Q.hm * (cs - Q.C_air(t)) for r, cs, t in zip(Rhist, Cshist, tt)]
85     rhs = np.trapezoid([a + b for a, b in zip(f1, f2)], tt)
86     out = dict(tstar=tstar, tab6=np.array(tab6), V1=(lhs, rhs, lhs - rhs))
87     if sample:
88         out['C'] = C
89     Q.NZ, Q.DZ, Q.zeta, Q.TEND, Q.CEND, Q.h, Q.hm = old
90     return out
91
92
93 if __name__ == '__main__':
94     print('==== A) 6h 行离散收敛 (dt=5s) =====')
95     r21 = row6h(1 / 20, 21)
96     r41 = row6h(1 / 40, 41)
97     r81 = row6h(1 / 80, 81)
98     d2141 = np.nanmax(np.abs(r21 - r41))
99     d4181 = np.nanmax(np.abs(r41 - r81))
100    print('21 vs 41 最大差 %.3e ; 41 vs 81 最大差 %.3e ; 比值 %.2f'
101          % (d2141, d4181, d2141 / d4181))
102    print('41 离散 6h 行:', np.round(r41, 4))
103    print('81 离散 6h 行:', np.round(r81, 4))
104
105    print('==== B) 时间收敛 (N=81, 6h 行, dt=5 vs 2.5) =====')
106    t5 = row6h(1 / 80, 81, 5.0)
107    t25 = row6h(1 / 80, 81, 2.5)
108    print('最大差 %.2e' % np.nanmax(np.abs(t5 - t25)))
109
110    print('==== C) Picard 两轮耦合残差 (N=81, dt=5) =====')
111    Q.NZ, Q.DZ = 81, 1 / 80
112    Q.zeta = np.arange(81) / 80
113    T, C = np.full(81, Q.T0), np.full(81, Q.C0)
114    for n in range(1, int(200000 / DT) + 1):
115        tn, tp = n * DT, (n - 1) * DT
116        tm = 0.5 * (tp + tn)
117        R = Q.RofT(tm)
118        Tn, Cn = T.copy(), C.copy()
119        Kk = Q.k4(C); Rc = Q.rho4(C) * Q.cp4(C)
120        T1 = Q.crank_step(Tn, Kk, Rc, R, Q.h * R / Kk[-1], Q.T_air(tp), Q.T_air(tn), DT)
121        Dc1 = Q.D4(C, T1 + 273.15)
122        C1 = Q.crank_step(Cn, Dc1, np.ones(81), R, Q.hm * R / Dc1[-1], Q.C_air(tp), Q.C_air(tn), DT)
123        Kk2 = Q.k4(C1); Rc2 = Q.rho4(C1) * Q.cp4(C1)
124        T2 = Q.crank_step(Tn, Kk2, Rc2, R, Q.h * R / Kk2[-1], Q.T_air(tp), Q.T_air(tn), DT)
125        Dc2 = Q.D4(C1, T2 + 273.15)
126        C2 = Q.crank_step(Cn, Dc2, np.ones(81), R, Q.hm * R / Dc2[-1], Q.C_air(tp), Q.C_air(tn), DT)
127        dC_pic = np.max(np.abs(C2 - C1)); dC_step = np.max(np.abs(C2 - Cn))
128        dT_pic = np.max(np.abs(T2 - T1)); dT_step = np.max(np.abs(T2 - Tn))
129        if tn in (3600, 21600, 72000, 172800):
130            print('t=%6ds: Picard |ΔC|=%.3e (步变化 %.3e, 比 %.1e) | ΔT|=%.3e (步变化 %.3e)'
131                  % (tn, dC_pic, dC_step, dC_pic / dC_step, dT_pic, dT_step))
132        T, C = T2, C2
133        if C.max() <= CTARGET:
134            break
135
136    print('==== D) 生产复算 N=81, dt=5 =====')
137    res81 = full_run(1 / 80, 81, tag='N=81 生产')
138    print('t* = %d s = %.4f h' % (res81['tstar'], res81['tstar'] / 3600))
139    print('表6 (N=81):')
140    for k, row in enumerate(res81['tab6']):
141        rr = ' '.join(' --' if np.isnan(v) else '%6.4f' % v for v in row)
142        trow = (k + 1) * 6.0 if k + 1 < len(res81['tab6']) else res81['tstar'] / 3600.0
143        print('%7.1fh |%s' % (trow, rr))

```

```

144 lhs, rhs, diff = res81['V1']
145 print('V1 守恒:  $\Sigma v\Delta C = .6e$  (...)dt=0.6e 差 0.2e (相对 0.1e)'
146       % (lhs, rhs, diff, abs(diff) / abs(lhs)))
147
148 print('==== E t* 离散收敛 21/41/81 ====')
149 res21 = full_run(1 / 20, 21, tag='N=21')
150 res41 = full_run(1 / 40, 41, tag='N=41')
151 t21, t41, t81 = res21['tstar'], res41['tstar'], res81['tstar']
152 print('t*(21)=0.4f h t*(41)=0.4f h t*(81)=0.4f h' % (t21 / 3600, t41 / 3600, t81 / 3600))
153 ext = (4 * t81 - t41) / 3
154 print('Richardson 外推(41,81): t*0.4f h ; (21,41): t*0.4f h'
155       % (ext / 3600, (4 * t41 - t21) / 3 / 3600))
156
157 print('==== F 消融实验: 附录4 物性 + 固定 R=2cm (N=81, dt=5) ====')
158 resFix = full_run(1 / 80, 81, fixed_R=True, tag='消融固定R')
159 print('t*_无收缩 = 0.4f h ; t*_有收缩 = 0.4f h ; 收缩贡献  $\Delta t = 0.4f h$  (0.1f%%)'
160       % (resFix['tstar'] / 3600, res81['tstar'] / 3600,
161          (resFix['tstar'] - res81['tstar']) / 3600,
162          (resFix['tstar'] - res81['tstar']) / resFix['tstar'] * 100))
163
164 print('==== G 灵敏度 (N=81, dt=5) ====')
165 sens = {}
166 for tag, kw in [('基线', {}), ('Ta+2', dict(Ta=52.165)), ('Ta-2', dict(Ta=48.165)),
167                ('Ca+20%', dict(Ca=0.04986 * 1.2)), ('Ca-20%', dict(Ca=0.04986 * 0.8)),
168                ('h+50%', dict(h=37.5)), ('h-50%', dict(h=12.5)),
169                ('hm+50%', dict(hm=1.2e-6)), ('hm-50%', dict(hm=4e-7))]:
170     if tag == '基线':
171         sens[tag] = res81['tstar']
172     else:
173         sens[tag] = full_run(1 / 80, 81, tag=tag, **kw)['tstar']
174     print('%-8s t* = 0.4f h ( $\Delta$  0.3f h)' % (tag, sens[tag] / 3600,
175         (sens[tag] - sens['基线']) / 3600))

```